

RESEARCH COLLABORATION PROGRAM

BETWEEN



National Engineering & Scientific Commission (NESCOM)

AND



National University of Science and Technology (NUST)

Project Title: Content Extraction and Filtering from Video Stream

Project Completion Report

Submitted by: Dr. Rabia Irfan, Assistant Professor, SEECS, NUST (Principal Investigator)

Signature: _____ **Dated:** _____

Abstract

The implemented methodology boasts a significant capability in real-time audio analysis, seamlessly integrated with the continuous capture of live video streams from YouTube. This real-time audio analysis is instrumental in identifying and extracting user-defined words within the audio content of the live stream. The process commences with the transformation of user-provided trigger words into synthesized speech using advanced text-to-speech (TTS) synthesis technique, facilitated by cutting-edge HiFiGANs—generative adversarial networks tailored specifically for audio generation. Renowned for their ability to produce high-fidelity speech waveforms closely resembling natural human speech, HiFiGANs ensure the synthesized speech is both clear and authentic-sounding. Subsequently, the synthesized speech is meticulously correlated with the real-time audio signal extracted from continuously downloaded YouTube video chunks, enabling a precise comparison of characteristics between the two. Upon detecting a significant correlation or match (above 70%) between the synthesized speech and the live audio signal, it indicates the presence of the specified word in the video stream's audio. In response, the methodology takes strategic action by saving video chunks—comprising 5 segments before and after the trigger word's occurrence—each lasting 55 seconds. This comprehensive approach ensures not only the capture of the detected word but also the pertinent contextual information surrounding it. Conversely, in instances where no substantial correlation or match is found, signalling the absence of the specified word in the live YouTube stream's audio, the methodology efficiently manages storage space by promptly deleting the associated video chunks for that specific timeframe. This proactive approach prevents the unnecessary accumulation of irrelevant data, optimizing resources for meaningful insights. Overall, the real-time audio analysis component significantly enhances the methodology's effectiveness in processing and analyzing live YouTube streams, enabling users to gain valuable insights and extract pertinent information from the ongoing streaming content.

Despite its success, the system faced certain limitations, including language restrictions, accuracy variability due to audio quality, and limited contextual understanding. Future work aims to address these limitations by extending multilingual support, integrating advanced speech recognition models and NLP techniques, incorporating deeper contextual analysis capabilities, enhancing the user interface, developing automated alerting and reporting features, and exploring integration with other data sources. By focusing on these areas, the project can evolve into a more robust, versatile, and user-friendly solution for real-time video stream analysis and word spotting.

Table of Contents

Chapter 1 – Introduction.....	5
1. About the Project.....	5
2. Project Implementation	6
3. Literature Review in a Nutshell.....	15
4. Report Purpose & Organization	18
Chapter 2 – Hardware Implementation.....	19
1. Hardware Specifications	19
2. Hardware Layout and Operations.....	25
3. Component Comparative Evaluation.....	27
Chapter 3 – Software Implementation	30
1. The Process Flow - An Overview	30
2. Thread#1 - Live Video Streams Capturing.....	33
3. Thread#2 – Video-to-Audio Conversion	34
4. Thread#3 - Speech Synthesis and Correlation Identification.....	36
Chapter 4 – Experimentation and Interfacing	42
1. Experiments and Results.....	42
2. The Graphical User Interface (GUI).....	49
3. Tools & Techniques Used.....	52
Chapter 5 – Conclusion	53
1. Summary	53
2. Contribution of the Project.....	56
3. Limitations & Future Work.....	57

Table of Figures

Figure 1 Video Monitoring System (VMS) Process Flow Chart	13
Figure 2 Hardware Components Used in the Project	20
Figure 3 Hardware Layout of Video Monitoring System (VMS).....	26
Figure 4 Flow Diagram of Signal-to-Signal Matching for in VMS	31
Figure 5 A Multi-threaded Approach for VMS Implementation	33
Figure 6 The HiFiGAN Architecture [Kong et al. (2021)]	36
Figure 7 Speech Synthesis with HiFi GAN	38
Figure 8 Example of Speech Synthesis given Text and Speakers	39
Figure 9 The Process of Correlation Identification	40
Figure 10 Speech-to-Text Conversion-based Approach	43
Figure 11 Urdu Dataset Speech with Transcription	44
Figure 12 Training of Wav2vec2 on Urdu Dataset	45
Figure 13 WER vs. CER	46
Figure 14 Urdu homophones Problem	46
Figure 15 The GUI of VMS	50
Figure 16 GUI Providing Real-time Feedback Provided to the User	51
Figure 17 GUI Providing Dual Channel Support	51
Figure 18 Development Stages of the Project	53

Chapter 1 – Introduction

This chapter provides an overview of what has already been done, some of which have already been reflected in the previous project report. This chapter will summarize the literature review that has been done to understand the state of the art. Finally, report organization is also discussed in this section.

1. About the Project

Project Title: Content Extraction and Filtering from Video Stream

Principal Investigator: Dr. Rabia Irfan (SEECS, NUST)

Co- Principal Investigators: Dr. Mureed Hussain; Dr. Muhammad Rizwan (NESCOM)

Research Assistant: Mr. Muhammad Babar Kamal (SEECS, NUST); Mr. Muhammad Hamza Tariq (SEECS, NUST)

Project Brief: Word searching and content extraction from textual data has been practiced for many years. Like textual data, audio in the video stream also carries useful information. Therefore, word searching from the audio content can be of paramount importance. In this project, a specific word's spoken time from the live audio/video stream will be localized. Secondly, the video content related to a specific word will be stored. The solution will be indigenous and comprise the portable box having a built-in AI module for word spotting and video stream capturing mechanisms.

Objectives & Milestones: The following are the objectives/milestones of this project (as given by NESCOM):

1. Developing a module that can identify the spoken time of a specific word from the video stream.
2. Developing a mechanism for recording the contents related to the searched word from the live video stream.
3. Building a box-based solution having a built-in AI module for word spotting and video stream capturing mechanism.

2. Project Implementation

For the development of the project, we have adopted a prototype-based approach in which the project requirements were collected and understood by showing up small chunks/parts of implementations. The project requirements were understood by repeatedly performing this activity. Based on this approach, the development of the project is divided into the following three stages:

- Stage 1- Initial Prototype Development
- Stage 2 - Integration & Prototype Refinement
- Stage 3 - Packaging and Deployment

We are done with the first two stages completely and Stage 3 is in progress. The details of the completed stages and in progress stage are discussed below.

2.1 Stage 1- Initial Prototype Development

Sub-tasks involved in this stage are as follows:

1. Live Video Stream Capturing
2. Speech Synthesis from Input Text
3. Correlation Identification
4. Setting up Jetson Nano

Below we summarize the tasks performed in these steps:

1. Live Video Stream Capturing:

The process of live stream video capturing involves extracting and downloading live video streams from platforms like YouTube using a structured methodology with open-source libraries. Initially, the *streamlink* library is utilized to pipe streams from services like YouTube into a video player, allowing local access and download. Next, the *m3u8* library is employed to load the associated playlist file, which contains URLs and timestamps of video chunks. The *get_stream* function extracts these URLs from the playlist, enabling the *dl_stream* function to download the video chunks. The *dl_stream* function iterates over the specified number of chunks, saving each as a separate file using the *urllib* library. To prevent duplicates, timestamps of each downloaded chunk are recorded. The captured video data consists of 5-second segments, continuously downloaded in

real-time until manually stopped. This continuous process ensures an uninterrupted and current collection of video content. Challenges such as network connectivity issues and efficient handling of large video chunks are addressed through exception handling and robust download management techniques. Overall, this methodology effectively uses *streamlink* and *m3u8* to manage live stream extraction, ensuring the successful download and storage of video chunks for further analysis and interaction.

2. Speech Synthesis from Input Text:

The project utilizes a sophisticated approach to transform user-provided trigger words into speech for real-time correlation with live YouTube videos. This process involves employing a *text-to-speech (TTS) synthesis* technique, which harnesses the capabilities of *HiFiGANs (High Fidelity Generative Adversarial Networks)*. *HiFiGANs* are specialized GAN architectures crafted specifically for audio generation, allowing for the creation of speech waveforms that closely mimic natural human speech. To implement this, a *pre-trained HiFiGAN model* is employed. When a user enters text, it undergoes encoding and is then fed into the *HiFiGAN* model. Subsequently, the model generates an audio waveform representing the synthesized speech corresponding to the user's input. *HiFiGANs* excel at capturing the subtle nuances and complexities of human speech, ensuring that the generated audio maintains high fidelity and sounds natural.

Once the speech synthesis is complete, the synthesized speech is correlated with the real-time audio signal extracted from the live YouTube videos. This correlation enables the identification and analysis of specific trigger words within the ongoing stream. By integrating *HiFiGANs* and *TTS* technology, the project significantly enhances the accuracy and effectiveness of processing and analyzing live YouTube videos, providing a more robust platform for real-time content understanding and interaction.

3. Correlation Identification:

The process of audio signal-to-signal matching involves two intricate steps, meticulously designed to facilitate effective correlation between user input and extracted audio signals from live YouTube video streams.

Firstly, the step of audio extraction and saving is performed, the step undertakes the crucial task of extracting audio from downloaded YouTube video chunks and systematically storing them as individual WAV files. This operation relies on a selection of essential libraries including *VideoFileClip* for accessing video content, *librosa* for audio processing, and *os* for file management, among others. The process begins by defining two critical paths: *audiopath* for saving extracted audio files and *videopath* for accessing downloaded video chunks. Each video chunk is systematically processed, where the current value of a count variable signifies the chunk being handled. The methodology meticulously verifies the existence of each video file before creating a *VideoFileClip* object to access its audio track. Subsequently, it determines the filename and path for the extracted audio file and employs a *write_audiofile()* function to save the audio as a WAV file. This process continues iteratively, incrementing the count variable with each iteration until all video chunks are processed. In case of any errors encountered during this process, the methodology gracefully handles them by printing error messages and attempting to re-read the video file.

Following the audio extraction phase, the correlation identification between audio signals step is performed. Here, the methodology undertakes a detailed comparison between the user-entered words, converted into speech, and the extracted audio signals from the live video chunks. Utilizing a carefully crafted formula, the correlation is calculated by multiplying elements of the audio signal arrays and summing the products. Matches between the user-entered words and extracted audio signals trigger the saving of corresponding video chunks for a specified duration of 55 seconds, ensuring the capture of the context surrounding the detected word. Conversely, in cases where no significant correlation or match is found, indicating the absence of the specified word in the live YouTube stream, associated video chunks are promptly deleted. This proactive approach not only optimizes storage space but also ensures that only relevant data is retained for further analysis and processing.

4. Setting up Jetson Nano:

Setting up the Jetson Nano board involves several crucial steps to prepare the hardware and software environment for optimal execution of the implemented methodology. Initially, the Jetson Nano board is readied by flashing the appropriate *JetPack* image onto the SD card, a process

facilitated by NVIDIA's comprehensive software package designed to include essential libraries, drivers, and tools for GPU-accelerated computing. Following the board preparation, the software development kit (SDK) components are installed, notably *CUDA* and *cuDNN*, vital for harnessing the GPU's parallel processing capabilities. *CUDA toolkit* furnishes a robust programming interface and tools for efficient GPU computing, while *cuDNN* optimizes deep neural network operations, particularly enhancing the performance of deep learning algorithms on NVIDIA GPUs. These components, sourced from the NVIDIA Developer website, are installed on the Jetson Nano board. Moreover, essential Python packages and libraries are installed to support various functionalities, including YouTube video downloading. Packages such as *cv2 (OpenCV)*, *pytube*, *m3u8*, and *streamlink* are obtained via the pip package manager, ensuring seamless integration with the Python environment. This meticulous setup ensures that the Jetson Nano SDK is equipped with the requisite software environment and GPU acceleration capabilities to execute tasks such as live stream extraction, audio processing, and speech synthesis effectively. By leveraging the computational power of the GPU, the Jetson Nano facilitates efficient processing, enabling the seamless execution of the implemented methodology.

At the end of the initial prototype development stage, we decided to scope the project to English language only. This is because of the challenges we faced in speech synthesis in the Urdu language, text-to-speech translation as well as audio signal-to-signal correlation identification. Next, we provide the details about the steps involved in the second stage of the project.

2.2. Stage 2- Integration & Prototype Refinement

The implementation of the prototype in the first stage significantly influenced the trajectory of the project's development. As we progressed, the focus shifted toward refining the prototype to achieve a more polished and robust outcome. This stage primarily entails several key sub-tasks aimed at enhancing the prototype's functionality and addressing any existing limitations. These sub-tasks include:

1. Refinement of Speech Synthesis
2. Hardware Components Connectivity
3. GUI Development
4. Hardware and Software Components Integration

Below we summarise the tasks performed in these steps:

1. Refinement of Speech Synthesis:

Initially, our project utilized the Google's *Text-to-Speech (GTTS)* library to synthesize speech, offering two variants to accommodate basic requirements for input text or text to be searched. However, as we refined the scope and aimed for higher precision and versatility, we integrated *HiFiGAN*, a state-of-the-art Generative Adversarial Network for speech synthesis for input text or text to be searched. *HiFiGAN* significantly enhances our capability by producing seven distinct variants and accents. These accents include: 2 types of US male and female accents, Canadian male accent, Scottish male accent and Indian male accent. This improvement allows for a more nuanced and accurate generation of speech, tailored to the specific accent and intonation required for different contexts.

Each of the seven accents generated by *HiFiGAN* is accompanied by a unique code, designed to facilitate precise accent matching. The different accents are crucial for increasing the signal-to-signal speech-matching accuracy between the text-to-speech synthesized words and the target words found in news channel videos. By capturing the subtle similarities in accents, our system ensures a higher fidelity in speech recognition and matching, thus enhancing the overall performance and reliability of our search and analysis processes. This not only improves the quality of synthesized speech but also significantly boosts the effectiveness of our project's objectives.

2. Hardware Components Connectivity:

As mentioned earlier Jetson Nano is the main processing unit for the implementation of this project. It requires connectivity with other components to ensure smooth working of the system to developed. Other than that we also have other hardware components that make up the whole system. The cooling fan, linked to the Jetson Nano, is essential for temperature management, this also helps in preventing overheating of the Jetson device. Another very important component is hard drive. Here the hard disk drive serves as the primary storage, holding captured video streams data from live video streams. The power supply unit is another vital component, providing reliable and accurate electrical power to the system. It not only supplies power but also protects against fluctuations, ensuring stable operation necessary for continuous data processing by Jetson Nano. Additionally, cables utilized for connecting different components, ensure robust connectivity,

enabling efficient communication and data exchange among various hardware elements. Last but not least, interface is established by connectivity keyboard and monitor to the Jetson Nano for operating the system. So by connecting these components, we developed the backbone of the system's architecture, allowing smooth coordination and interaction between various components.

3. GUI Development:

For the Graphical User Interface (GUI) development, we've created a user-friendly interface using *Tkinter*, a Python library. *Tkinter* helps us design windows, buttons, and text fields that users can interact with easily. In our GUI, users can enter search words in a text field. Users have also to provide the URL of a YouTube channel where they want to search for those words. The interface layout is kept simple and intuitive, with clear labels and input fields to guide users. When users input their search words and YouTube channel URL, they can click on a “Start” button to initiate the process.

Behind the scenes, the GUI interacts with backend algorithms to process the user's input and fetch relevant data from the specified YouTube channel. Overall, the GUI development ensures that users can comfortably input their search criteria and interact with the system effortlessly, facilitating a smooth user experience.

4. Hardware and Software Components Integration:

In bringing together hardware and software, our project carefully merges different parts to make a working system that we named as “Video Monitoring System (VMS)”. The hardware, like the NVIDIA Jetson Nano, cooling fan, hard disk drive, and power supply unit, are all joined together to create the physical setup. Each piece is connected and set up in a way that makes sure everything works well and doesn't break down. The Jetson Nano leads the way, making sure all the hardware and software work together smoothly.

The Jetson Nano uses software including the *JetPack SDK* and also the customised python scripts we wrote to do tasks like capturing live video, making speech from text, and understanding audio signals. These software bits are carefully integrated into the hardware, so they can use the Jetson Nano's computing power to give us useful information in real-time.

The overall flow of the steps involved in the monitoring of video streams are depicted in Figure 1. By putting hardware and software together in the right way, our project VMS builds a strong and flexible system. It helps us search for specific words and get information from live audio and video streams. The in depth explanation of the system with respect to hardware components is given in Chapter 2, whereas Chapter 3 explains the working of the software aspect of the system in depth.

2.3. Stage 3- Packaging and Deployment

This is the last stage of the project and is currently progressing. We have already completed implementation of a workable system with both software and hardware components enabled and running. Now, we are working towards the production of the final system for which we are in a process of purchasing the components. This stage mainly involves the following steps:

1. Equipment Purchase
2. Casing of Hardware Components
3. Project Documentation
4. Project Deployment

We perform the following tasks to produce the final hardware box containing the “Video Monitoring System (VMS)”:

1. Equipment Purchase:

To ensure the smooth completion of the final stage, the first step is procuring the necessary equipment for the final development of the Video Monitoring System (VMS). By acquiring high-quality components, we ensure the system's longevity and reliability in real-world applications. The purchase process has started at NUST and the following components are being ordered:

1. Jetson Orin
2. Memory Card, 64 GB
3. HDMI to HDMI Cable
4. 40mm x 10mm 0.12A 2Pin 5V DC Brushless Sleeve Bearing Cooling Fan
5. 1TB Hard Disk, SSD preferred with hard disk to USB converter

The status of the purchase case as received on 26 July, 2024 is attached as Annex A.

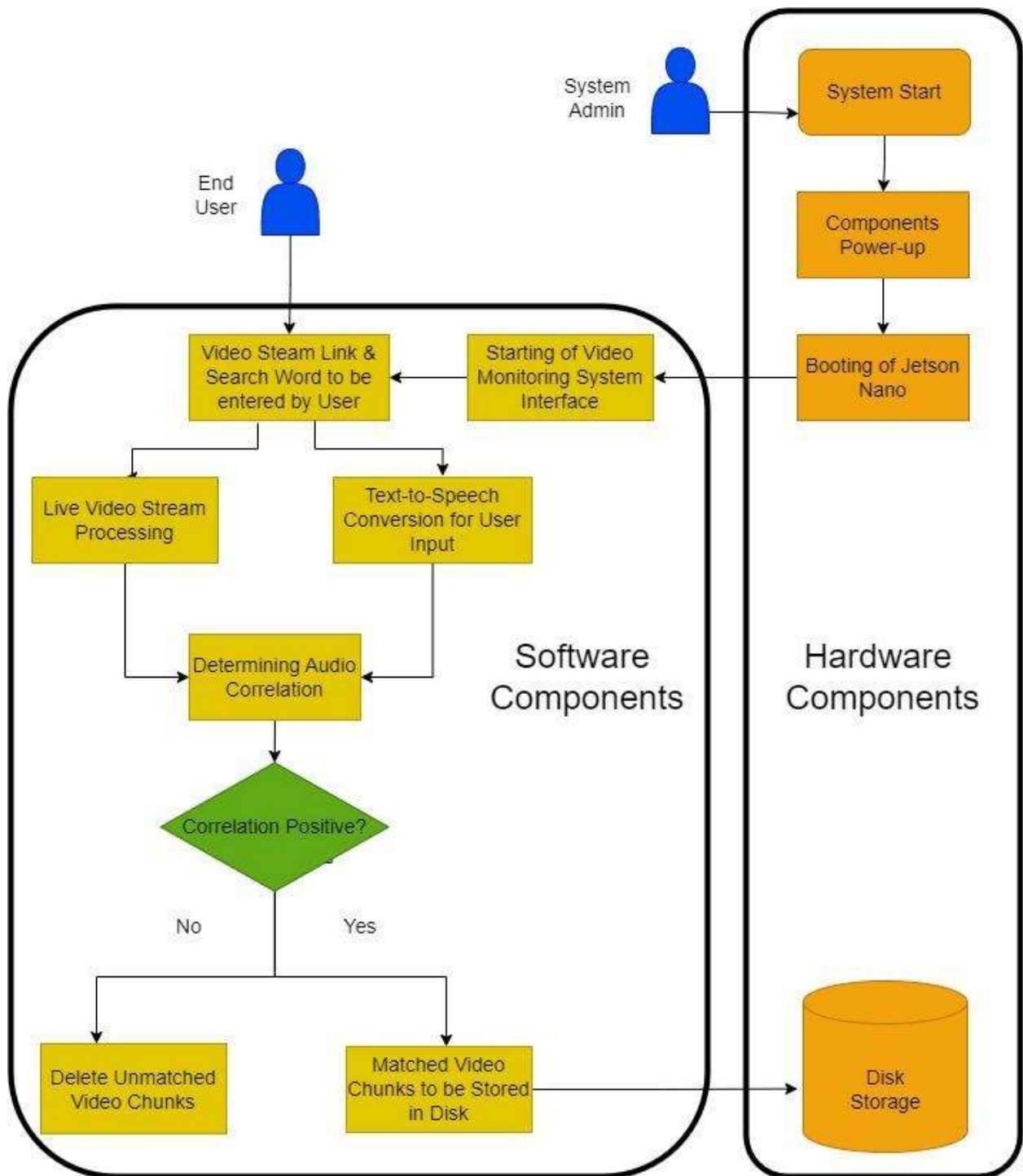


Figure 1 Video Monitoring System (VMS) Process Flow Chart

2. Casing of Hardware Components:

Once the equipment will be procured, the next step will be to case the hardware components. This involves assembling all parts into a secure, robust, and user-friendly enclosure. The casing process includes:

- *Mounting the Jetson Device and Other Components:* Securely mounting the Jetson device, cooling fans, hard drives, and power supply unit within the enclosure, ensuring the algorithms should be in working form within the Jetson device.
- *Cable Management:* Organizing and securing cables to prevent damage and ensure efficient airflow.
- *Cooling Solutions:* Implementing effective cooling solutions to maintain optimal operating temperatures for the hardware.
- *Access Points:* Designing and installing access points for peripheral connections such as USB ports, HDMI, and power inputs.

The casing process not only protects the hardware but also enhances the system's portability and ease of use.

3. Project Documentation:

Comprehensive project documentation is crucial for the successful deployment and future maintenance of the VMS. Depending upon the requirement of NESCOM, the documentation may include:

- *System Architecture:* Detailed diagrams and descriptions of the hardware and software components and their interactions.
- *Installation Guide:* Step-by-step instructions for setting up the system, including hardware assembly and software installation.
- *Source Code Documentation:* Detailed comments and explanations within the code to facilitate understanding and future development.
- *User Manual:* Clear and concise guidelines for end-users to operate the VMS, including how to input search words, start video processing, and access results.

This final project report fulfils most of the above requirements, remaining necessary documentations as per the requirements of NESCOM will be completed before closing the project.

In short, by providing thorough documentation, we ensure that users and developers can easily understand, use, and maintain the system.

4. Project Deployment:

The final step in Stage 3 is the deployment of the VMS. Depending upon the requirement of NESCOM, this process may include the following:

- *System Testing:* Conducting rigorous tests to ensure all components are functioning correctly and the system performs as expected under various conditions.
- *Deployment Setup:* Installing the VMS in the intended environment, whether it be a server room, an office, or another location.
- *User Training:* Providing training sessions for users to familiarize them with the system's functionality and usage if required.
- *Monitoring and Support:* Establishing a monitoring and support framework to address any issues that may arise post-deployment.

By following these steps, we ensure that the VMS is effectively deployed, operational, and ready to meet user needs in a real-world setting.

3. Literature Review in a Nutshell

Extensive literature review has been done in the first stage of the project and that has already been added in the first progress report. However, here we will present summary of the already done literature review for the purpose of connecting with the state of the art related to the work done in the project.

Our literature review has two main parts. Firstly, we summarize the speech processing related approaches and secondly, we summarize the speech synthesis approaches that has been reviewed during the process of implementation.

3.1 Speech Processing Approaches

We compared two approaches of speech processing: Speech-to-Text Conversion and Speech Signal Processing. Below, we will be briefly discussing both of the two approaches concluding that with the critical analysis of the two approaches:

1. Speech-to-Text Conversion:

Speech-to-text conversion-based approaches convert speech/audio to text, which is then used for further processing. Speech-to-text conversion, also known as automatic speech recognition (ASR), has seen significant advancements with the rise of deep learning techniques. Deep Neural Networks (DNNs) and Recurrent Neural Networks (RNNs) have played crucial roles in improving ASR system accuracy and performance. DNNs excel in capturing intricate speech feature patterns and have become a standard in ASR when combined with Hidden Markov Models (HMMs). Hinton et al. (2012) introduced Deep Belief Networks (DBNs) for pre-training DNN-HMM systems, resulting in substantial accuracy enhancements. Recurrent Neural Networks, especially Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures, have effectively modelled sequential speech data dependencies. Graves et al. (2013) and Sak et al. (2015) demonstrated significant improvements in phoneme recognition and end-to-end speech recognition using Bidirectional LSTM (Bi-LSTM) and sequence-to-sequence LSTM models, respectively. Attention mechanisms have emerged as crucial in ASR, with models like Listen, Attend, and Spell (LAS) by Chan et al. (2016) using attention to focus on relevant input sequences, enhancing transcription accuracy. Transformer-based models, such as the Conformer model by Gulati et al. (2020), combine Convolutional Neural Networks (CNNs) and self-attention to capture both local and global contextual information, outperforming traditional RNN-based models. Additionally, transfer learning and data augmentation techniques, as highlighted by Ko et al. (2017), have improved ASR performance, especially in low-resource scenarios, by fine-tuning pre-trained models and augmenting training data with variations to simulate real-world conditions. In summary, deep learning techniques, including DNNs, RNNs, attention mechanisms, and transformer-based models, have revolutionized speech-to-text conversion, achieving state-of-the-art results and paving the way for more accurate and robust ASR applications.

2. Speech Signal Processing:

Recent advancements in deep learning and signal processing have led to various innovative approaches in the field of speech signal processing. Feature extraction is a very crucial component of applications dealing with audio signals, with spectrogram-based features like Mel-Frequency Cepstral Coefficients (MFCCs) and Log-Mel Spectrograms being widely used. Kumar et al. (2020) introduced Graph Spectrogram Convolutional Networks (GSCNs), which use Graph Convolutions

on Log-Mel Spectrograms to improve matching performance. Deep learning architectures, such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), have also been successful. Choi et al. (2017) employed a Siamese CNN for efficient similarity comparison, while Lee et al. (2019) proposed a Siamese LSTM network for improved temporal dependency learning. Metric learning, particularly triplet loss, has been effective in learning discriminative embeddings for audio matching, as demonstrated by Chen et al. (2019). Self-supervised learning, which leverages unlabeled data, has also shown promise. Kong et al. (2021) introduced the BYOL (Bootstrap Your Own Latent) framework, achieving impressive performance through representation learning. Additionally, Graph Neural Networks (GNNs) have gained popularity for their ability to capture complex relationships between audio signals, with Jin et al. (2020) proposing a GNN-based approach for effective audio matching. Overall, these advancements in feature extraction, deep learning architectures, metric learning, self-supervised learning, and GNNs have significantly improved the accuracy and efficiency of audio signal matching systems.

3. Comparative Analysis:

Speech-to-text conversion and speech signal processing are the two distinct approaches for speech processing, each with its own advantages and disadvantages. Speech-to-text conversion offers semantic understanding and language processing capabilities, allowing for the use of text-based search algorithms and natural language processing techniques. This method is accessible and interpretable for users but suffers from the loss of fine-grained acoustic information and depends heavily on the accuracy of speech recognition systems. Conversely, speech signal processing retains rich acoustic information and is robust to language variations and noise, making it suitable for multilingual or noisy environments. However, it lacks semantic understanding, can be less interpretable, and is often computationally intensive. The choice between these methods depends on the application requirements, resources, and the nature of the audio data. Given the need for a language-independent solution, the speech signal processing approach has been selected for the implementation.

3.2 Speech Synthesis Approaches

Speech synthesis, or Text-to-Speech (TTS), generates natural-sounding speech from text using various approaches. Traditional concatenative synthesis involves concatenating pre-recorded

speech segments but requires extensive manual efforts (Klatt, 1987). Statistical parametric synthesis, utilizing models like Hidden Markov Models (HMMs), provides high-quality speech with improved naturalness (Zen et al., 2009). However, deep learning techniques have revolutionized the field, with models such as Generative Adversarial Networks (GANs) and Transformer-based systems like Tacotron 2 producing highly natural and expressive speech (Shen et al., 2018). Neural vocoders like WaveNet generate high-fidelity waveforms, capturing detailed nuances (Van den Oord et al., 2016). Recent advancements focus on controlling speech expressiveness using Variational Auto Encoders (VAEs) and enabling multilingual synthesis (Skerry-Ryan et al., 2018; Wang et al., 2020). While traditional methods are reliable, GAN-based models offer improved naturalness, reduced dependency on complex features, and customizable synthesis, making them suitable for robust, adaptable speech generation. Therefore, the project adopts GAN-based TTS for its implementation, leveraging these benefits for high-quality speech synthesis.

4. Report Purpose & Organization

This is the second and the final progress report of the project. The purpose of this document is to present the implementation of the project and details of the work done in the project from Jan 23 to Jun 24. The organization of the document is as follows:

- *Chapter 2 - Hardware Implementation:* This chapter describes the hardware components and their assembly within the project's enclosure. It details the equipment used and the methodology adopted for the implementation of the hardware system responsible for video extraction, processing, and storage.
- *Chapter 3 - Software Implementation:* This chapter outlines the finalized process flow of the system, developed according to the requirements specified by the NESCOM team. It covers the details about the software implementation, including the specific algorithms utilized in the project.
- *Chapter 4 - Experimentation and Interfacing:* This chapter discusses the various approaches tested during the project's development and presents the results obtained. It also includes details on the development of the graphical user interface (GUI) and its integration with the system.
- *Chapter 5 - Conclusion:* This chapter provides a summary of the project, highlighting its contributions. It also addresses the project's limitations and proposes directions for future work.

Chapter 2 – Hardware Implementation

This chapter provides details of the hardware implementation of the project. In this chapter, first, we will discuss the specifications of different hardware components used in the project. The connectivity of these components to make them operational will be discussed next. In the end, we will perform a comparative analysis of various options available for performing the main processing task in the project.

1. Hardware Specifications

In this section, we provide a comprehensive overview of the components employed in the project along with their respective specifications. The selection of appropriate components is vital to ensure the functionality, efficiency, and reliability of the Video Monitoring System (VMS), which is developed as a result of this project. Each component plays a crucial role in contributing to the overall performance of the system, from data processing to storage and power management. By detailing the specifications of each component, we aim to offer insight into the technical aspects of the hardware setup, facilitating a clear understanding of the system architecture and its capabilities. The following components, as shown in Figure 2, are used for the implementation of the hardware part of this project:

1. NVIDIA Jetson Nano
2. 12V DC Cooling Fan
3. Hard Disk
4. Power Supply, Necessary Cables & Interfaces



Figure 2 Hardware Components Used in the Project

Below, we will provide specifications of the components used:

1. NVIDIA Jetson Nano:

The NVIDIA Jetson Nano is a compact and powerful computer designed to accelerate Artificial Intelligence (AI) and Machine Learning (ML) applications. The Jetson Nano's small form factor and robust capabilities make it suitable for a wide range of applications, from robotics and Internet of Things (IoT) to Computer Vision and Natural Language Processing. Due to this capability of Jetson Nano, we selected it for the development of a prototype in our project. The main tasks here are the capturing and processing of videos from the live streams, speech synthesis from the input text, and audio signal-to-signal matching between the speech obtained from the videos and the text entered by the end user. With its easy-to-use development environment and support for popular AI frameworks, the Jetson Nano helped us in the rapid prototyping and deployment of models for content extraction and speech synthesis tasks in our Video Monitoring System (VMS). Below we provide some details about the specifications of components, in general, inside the Jetson Nano kit that we employed in the implementation of VMS.

- *CPU*: The CPU used is Quad-Core ARM Cortex-A57 MPCore processor. The Jetson Nano's CPU provides ample processing power for multi-threaded applications, ensuring smooth and efficient performance.
- *GPU*: The GPU used is 128 NVIDIA Cuda Core maxwell architecture. Equipped with a powerful GPU, the Jetson Nano excels in parallel processing, making it perfect for tasks performed by the VMS.
- *Memory*: 4 GB 64-bit LPDDR4 RAM is used in the Jetson Nano kit that we had. With 4 GB of high-speed RAM, the Jetson Nano can handle complex models and large datasets, ensuring that applications run smoothly and efficiently, however, we faced that the system throughput needs to be improved, plus some of the graphic libraries for providing the enhanced user interface requires a better capacity of RAM. So we decided to opt for a larger capacity RAM in the final product.
- *Storage*: MicroSD card slot for storage expansion: The inclusion of a MicroSD card slot allows for easy and flexible storage expansion, providing the capacity to store extensive video datasets and backend models.
- *Connectivity*: Gigabit Ethernet ensures fast and reliable wired network connections, while Wi-Fi and Bluetooth provide convenient wireless connectivity options for a wide range of peripherals and networked devices, which is also a requirement of our system.
- *Interfaces*: The interfaces provided by the Jetson Nano are: USB 3.0, USB 2.0, HDMI, display port, CSI camera interface. USB 3.0 and USB 2.0 ports offer versatile connectivity for peripherals such as keyboards, mice, and external storage devices. They are used in our system to provide interface to the end user who wants to search for a particular word in the video. HDMI and display port output support high-definition video output, which is required in our system as the output of VMS is the video extracts related to the search words. The CSI camera interface supports direct connection to compatible cameras, facilitating projects that involve computer vision and real-time video analysis. However, this interface is not much of a use for the case of VMS.

In addition to that, the following features of the Jetson Nano has made it a suitable choice for us to prototype the Video Monitoring System (VMS):

- *Developer-Friendly*: The NVIDIA Jetson Nano's compatible with NVIDIA's JetPack SDK support for popular AI frameworks such as TensorFlow, PyTorch, and Caffe, along with its

extensive documentation and community support, was invaluable in implementing our Video Monitoring System.

- *Energy-Efficient:* Despite its powerful performance, the Jetson Nano is designed to be energy-efficient, making it suitable for battery-powered applications and reducing operational costs in continuous deployment scenarios.

Jetson Nano is the main component of the hardware box running the VMS. Other support hardware components are discussed next.

2. 12V DC Cooling Fan:

A cooling fan is employed to maintain optimal operating temperatures for the NVIDIA Jetson Nano, ensuring reliable performance and longevity of the device. Effective cooling is crucial for preventing overheating, which can lead to thermal throttling and reduced computational efficiency. This cooling fan is specifically designed to provide efficient airflow, keeping the Jetson Nano and its components cool even during intensive AI and machine learning tasks. The cooling fan that we used in our project has the following specifications:

- *Voltage:* The fan operates at a standard 12 Volts (V) of Direct Current (DC), ensuring compatibility with the power requirements of the NVIDIA Jetson Nano and other similar electronic devices.
- *Size:* With a standard 80mm dimension, the fan fits easily into most enclosures designed for the Jetson Nano, providing flexibility in design and installation.

The fan that we used in the prototype development has a problem of noise. Therefore, we targeted to opt for a fan with the following features in the final product:

- *Airflow Efficiency:* The cooling fan must be designed to deliver optimal airflow, ensuring that heat is effectively dissipated from the Jetson Nano's critical components. This will help in maintaining stable operating temperatures during prolonged use and under heavy computational loads.
- *Noise Level:* The cooling fan must be engineered to operate quietly, and must minimize noise.
- *Easy Installation:* The fan must come with standard mounting holes and connectors, facilitating straightforward installation onto the Jetson Nano or its enclosure.

3. Hard Disk for Data Storage:

A Hard Disk Drive (HDD) is employed to store the vast amounts of data collected from video streams. With its high storage capacity and reliable performance, the HDD is essential for archiving video content, enabling efficient retrieval and analysis of stored data. This storage solution ensures that the data remains secure and accessible, supporting the demanding requirements of video content extraction. HDD with following specifications was employed in the prototype development stage of the project:

- **Capacity:** The HDD that we used provides 1 terabyte of storage space, which is ample for storing extensive video data at the prototype stage, allowing for prolonged recording periods and the retention of high-definition video files.
- **Interface:** Equipped with a SATA interface, the hard disk drive offers high-speed data transfer rates and compatibility with a wide range of systems and motherboards. This ensures seamless integration and reliable performance.

In order to further improve the system efficiency, we will opt for Solid State Drive (SSD) in the implementation of the final product. SSD is a storage device that uses flash memory to store data. It has no moving parts, which allows for faster data access and higher durability compared to HDDs. SSD are mostly employed in:

1. Applications where fast boot and load times are crucial, as in the case of VMS.
2. High-performance computing tasks such as dealing with video related task, as is done by VMS.
3. Portable storage solutions where durability and shock resistance are important, as VMS will be a box-based solution, so this is indeed very closely related to our need.
4. Environments where power efficiency is a priority, which is also desirable for our VMS.

All these cases are closely related to the requirement of our VMS. Table 1 further compares the two types of disk storage:

Table 1 HDD vs. SSD

Feature	Hard Disk Drive (HDD)	Solid State Drive (SSD)
Capacity	Higher capacities available (1TB to 10TB+)	Increasing capacities (up to 4TB+), but generally lower than HDDs
Cost	Lower cost per gigabyte	Higher cost per gigabyte, but decreasing over time

Speed	Slower read/write speeds (80-160 MB/s)	Significantly faster read/write speeds (200-550 MB/s for SATA, up to 3500 MB/s+ for NVMe)
Durability	Prone to mechanical failure, less shock-resistant	More durable, resistant to physical shock
Power Consumption	Higher power consumption	Lower power consumption
Form Factor	3.5-inch, 2.5-inch	2.5-inch, M.2, PCIe
Use Cases	Bulk storage, cost-sensitive applications	Performance-critical tasks, portable storage

In short, HDDs offer larger storage capacities at a lower cost, making them ideal for bulk storage and cost-sensitive applications. SSDs, on the other hand, provide significantly faster speeds, greater durability, and lower power consumption, making it a suitable choice for our case.

4. Power Supply, Necessary Cables and Interfaces:

A power supply unit is essential for delivering the necessary electrical power to the system components, including the cooling fan and the NVIDIA Jetson Nano. By providing stable and regulated power, the unit ensures that these components operate efficiently and reliably. Power supply with the following specifications was utilized for the prototype development:

- **Voltage:** The power supply outputs both 12 V of DC and 5 V of DC, catering to the different voltage requirements of both the cooling fan and the Jetson Nano. This is a customized power supply given by NESCOM to fulfil the task of prototype development and the same will be used for the production of the final product.
- **Current:** With a current rating of 2 Amperes, the power supply is capable of supplying sufficient power to the connected components, ensuring that both the Jetson Nano and the cooling fan receive adequate current for their operations.

The box-based solution produced for VMS is operated via GUI and the interface to which is possible by using a display screen and a keyboard.

Last but not least, various cables are utilized for power supply and display connections, ensuring seamless integration and communication between components. These include:

- *Power Cable*: Used to connect the power supply to the Jetson Nano and cooling fan, ensuring that both receive the necessary electrical power for operation.
- *Display Cable*: Used to connect the Jetson Nano to a display monitor, allowing for visual output and interaction with the system.
- *Data Cable*: Used for connecting the hard disk drive for data storage, facilitating the transfer and storage of video content and other data.

Using the above components specification, VMS prototype has been built. Next, we will explain the layout of these hardware components and the operations performed by them.

2. Hardware Layout and Operations

In this section, we provide an overview of the system operation and hardware layout. The hardware layout depicts the arrangement of components within the system, highlighting their positions and connections. Additionally, the operations outline the functionalities and tasks performed by each component during system operation.

2.1 Hardware Layout

The hardware layout of the system is shown in Figure 3 below:

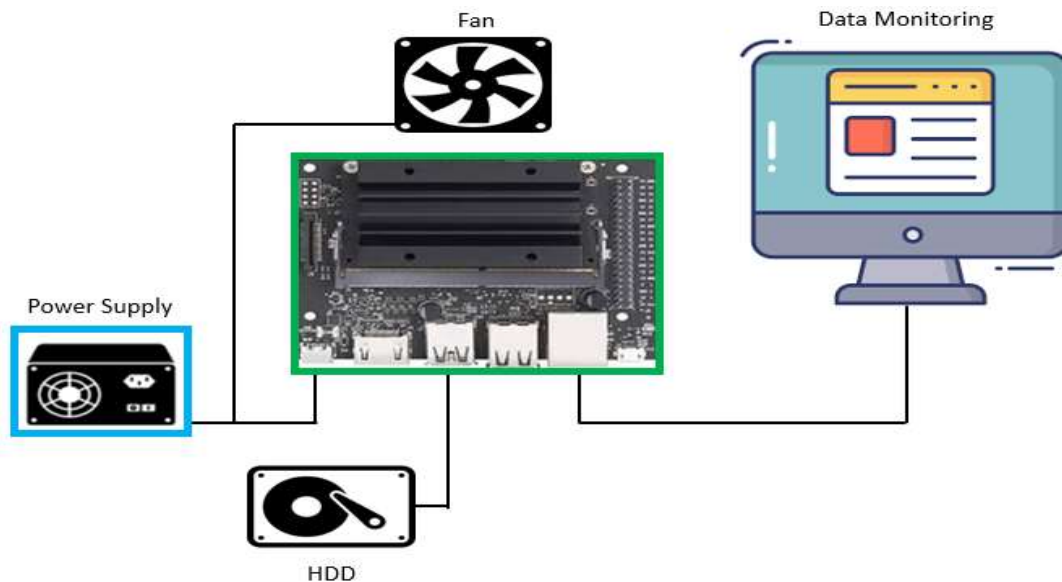


Figure 3 Hardware Layout of Video Monitoring System (VMS)

It comprises the following components arranged in a compact and efficient manner:

1. **NVIDIA Jetson Nano:** Positioned at the center of the layout, the NVIDIA Jetson Nano serves as the central processing unit for the system. It is connected to other components for data processing and management.
2. **12 V DC Cooling Fan:** Located adjacent to the NVIDIA Jetson Nano, the 12 V DC cooling fan is mounted to ensure efficient heat dissipation and maintain optimal operating temperatures.
3. **Hard Disk Drive (HDD):** Positioned on bottom side of the layout, the hard disk drive (HDD) serves as the primary storage device for storing data collected from the video streams. It is connected to the Jetson Nano for data transfer.
4. **Power Supply Unit:** Placed on the left side of the layout, the power supply unit provides electrical power to both the cooling fan and the NVIDIA Jetson Nano. It is connected to the respective components via power cables.
5. **Display Screen:** A display screen is mounted on the right side of the layout, the display screen is used to interact with the system and is also used for data monitoring in real time.
6. **Necessary Cables:** Various cables, including power cables, display cables, and data cables, are neatly arranged and connected to the corresponding components to ensure seamless operation and connectivity.

2.2 Operations

The system is designed to function seamlessly from power-up to the execution of video processing tasks. The operational details of the system are as follows:

1. **Power Supply Unit Activation:** Upon power-up, the unit supplies stable and regulated electrical power to the system components, including the NVIDIA Jetson Nano and the 12 V DC cooling fan. This ensures that all essential hardware is energized and ready for operation.
2. **Continuous Cooling Operation:** The 12 V DC cooling fan operates continuously to manage the thermal output generated by the Jetson Nano during intensive processing tasks. By dissipating heat efficiently, the fan ensures that the system's temperature remains within safe operational limits, thus preventing overheating and ensuring optimal performance.

3. **NVIDIA Jetson Nano Initialization:** The NVIDIA Jetson Nano initializes by booting up its operating system and loading the necessary software modules. These modules are specifically designed for handling tasks such as capturing live video streams, processing audio signals, and executing AI algorithms for trigger word detection.
4. **Live Video Stream Processing:** The Jetson Nano captures live video streams using Python library. These streams are then processed in real-time, the system processes incoming audio signals from the video streams, converting video chunks into audio format to prepare them for matching and analysis.
5. **Speech Synthesis of Trigger Words:** The transformation of user-provided trigger words into synthesized speech using a text-to-speech (TTS) synthesis technique is performed. This is achieved by employing state-of-the-art HiFiGANs, generative adversarial networks designed specifically for audio generation.
6. **Correlation Identification:** The audio processing module detects user-defined trigger words within the audio stream. Once a trigger word is identified, the system correlates this audio event with the corresponding segments of the video stream.
7. **Video Content Extraction:** Upon identifying the trigger words, the system extracts the relevant segments of the video content. These segments are carefully selected to ensure that they encompass the context and details surrounding the trigger events.
8. **Storage in Hard Disk Drive (HDD):** The extracted video content is then stored securely in a 1TB SATA hard disk drive. This storage solution provides ample capacity for archiving large volumes of video data, ensuring that all important content is preserved for future analysis and retrieval.
9. **User Interface and Monitoring:** The system includes a user-friendly interface that allows users to configure settings, define trigger words, and monitor the status of ongoing processes. This interface provides real-time feedback and control over the system's operations.

3. Component Comparative Evaluation

In our comparative evaluation of different Jetson devices for our final product development, including the Jetson Nano, Jetson TX2, Jetson Xavier, and Orin Nano, we have analyzed various key parameters to determine their suitability for our project requirements, as given in

Table 2. Each Jetson device offers unique specifications in terms of CPU, GPU, AI acceleration, memory, storage, USB connectivity, dimensions, AI performance, and power consumption.

Table 2 Comparison of different Jetson Variants

	Jetson Nano	Jetson TX2	Jetson Xavier	Orin Nano 8GB
CPU	Quad-core Arm A57 @ 1.43 GHz	Hexa-core processor with 2x NVIDIA Denver 2 64-Bit CPU, 4x ARM Cortex-A57 cores	6-core NVIDIA Carmel ARM v8.2 64-bit CPU 6 MB L2 + 4 MB L3 cache	6-core Arm® Cortex®-A78AE v8.2 64-bit CPU 1.5MB L2 + 4MB L3
GPU	128-core Maxwell GPU	256-core Pascal GPU	Volta GPU with 384 NVIDIA CUDA cores and 48 Tensor cores	1024-core NVIDIA Ampere architecture GPU with 32 Tensor Cores
Memory	4 GB 64-bit LPDDR4 @ 25.6 GB/s	8GB 128-bit LPDDR4 @ 59.7 GB/s	8GB 128-bit LPDDR4x @ 51.2GB/s	8GB 128-bit LPDDR5 68 GB/s
Storage	MicroSD slot	32GB eMMC flash SD card slot SATA	32GB eMMC flash SD card slot SATA	- (Supports external NVMe)
USB	4x USB 3.0 1x Micro USB 2.0 device	1x USB 3.0 1x Micro USB 2.0	4x USB 3.1 1x Micro USB 2.0	USB Type-A: 4x USB 3.2 Gen2/ USB Type-C (UFP)
Dimensions	100 x 80 x 26 mm	170x170mm (Mini-ITX)	103 x 90.5 x 31 mm	100 mm x 79 mmx 21 mm
AI Performance	472 GFLOPs	1.3 TFLOPS	21 TOPS	40 TOPS
Power Consumption	5-10W	7.5-15W	10-15W	10W / 15W / 25W

The Jetson Nano, with its quad-core Arm A57 CPU and 128-core Maxwell GPU, provides sufficient computational power for basic applications while consuming relatively lower power (5-10W). We have used

this device for our initial prototype development. However, it lacks dedicated AI acceleration engines compared to the more advanced models, which limits its capability for complex AI tasks. On the other hand, the Jetson Xavier features a 6-core NVIDIA Carmel ARM CPU and a Volta GPU with 384 CUDA cores and 48 Tensor cores, offering significantly higher AI performance (21 TOPS) but with increased power consumption (10-15W). Its advanced AI capabilities make it suitable for complex AI tasks but may be overkill for applications that do not require such high levels of performance. The Jetson TX2 lies between the Nano and Xavier in terms of performance and power consumption. It features a hexa-core processor with a Pascal GPU and offers a balance between computational power and energy efficiency. Its 1.3 TFLOPS AI performance makes it a versatile choice, but it may still be more than necessary for less demanding applications.

The Orin Nano, however, stands out as the most advanced among the evaluated devices. It features a 6-core Arm Cortex-A78AE CPU and a 1024-core NVIDIA Ampere architecture GPU with 32 Tensor Cores. This combination provides an impressive 40 TOPS of AI performance, making it ideal for high-intensity AI and real-time processing tasks. Additionally, it offers superior memory bandwidth with 8GB 128-bit LPDDR5 at 68 GB/s and supports multiple power consumption profiles (10W, 15W, 25W), providing flexibility for different deployment scenarios. Moreover, the Orin Nano supports modern connectivity options, including 4x USB 3.2 Gen2 and USB Type-C (UFP), and can accommodate external NVMe storage, making it highly adaptable for various applications.

Overall, our comparative evaluation highlights that while the Jetson Nano, TX2, and Xavier each have their strengths, the Orin Nano is superior in terms of AI performance, memory bandwidth, and connectivity options. This advanced capability, combined with flexible power consumption profiles, makes the Orin Nano the best choice for our final product development, ensuring it meets our end-user requirements for high performance and versatility. The 8GB RAM also resolve issues of slow throughput that we faced during testing of the prototype.

Next, chapter explains the software implementation of the VMS.

Chapter 3 – Software Implementation

This chapter provides details of the software implementation of the project. Basically in this chapter, we will introduce the algorithms behind the Video Monitoring System (VMS) functionality. We adopted a multi-threaded approach towards the implementation of algorithms behind VMS and this chapter explains the working of each of the involved threads.

Let's explore the software aspect of the VMS in detail.

1. The Process Flow - An Overview

The key capabilities of the implemented methodology lies in its ability to perform real-time audio analysis in conjunction with the continuous capture of live video streams from YouTube. This real-time audio analysis is a crucial component that enables the identification and extraction of specific user-defined words within the audio content of the live stream. The process begins with the transformation of user-provided trigger words into synthesized speech using a text-to-speech (TTS) synthesis technique. This is achieved by employing state-of-the-art HiFiGANs, generative adversarial networks designed specifically for audio generation. HiFiGANs excel in producing high-quality speech waveforms that closely mimic natural human speech, ensuring that the synthesized speech is clear and natural-sounding.

Once the user's input text is transformed into synthesized speech, it is correlated with the real-time audio signal extracted from the continuously downloaded YouTube video chunks. This correlation process involves comparing the characteristics of the synthesized speech with the features of the live audio stream. When a significant correlation or match is detected (greater than 70%) between the synthesized speech and the live audio signal, it signifies that the specified word is present in the video stream's audio. In response, the methodology takes action by saving the video chunks of 55 seconds in length, i.e., 5 video chunks before the occurrence of trigger word detection and 5 video chunks after the occurrence of trigger word detection. This ensures that not only the detected word but also the relevant context surrounding it is captured. This approach ensures that the extracted data contains meaningful and informative content related to the identified word or

phrase. Conversely, if no significant correlation or match is found between the synthesized speech and the live audio signal, it indicates the absence of the specified word in the live YouTube stream's audio. In such cases, the methodology efficiently manages storage space by deleting the video chunks associated with that specific timeframe, preventing unnecessary accumulation of data that does not contain the desired word. The real-time audio analysis component of the methodology enhances its overall effectiveness in processing and analyzing live YouTube streams. By identifying and capturing user-defined words in real-time, it empowers users to gain insights and extract valuable information from live streaming content. The process flow diagram of the system is shown below in Figure 4:

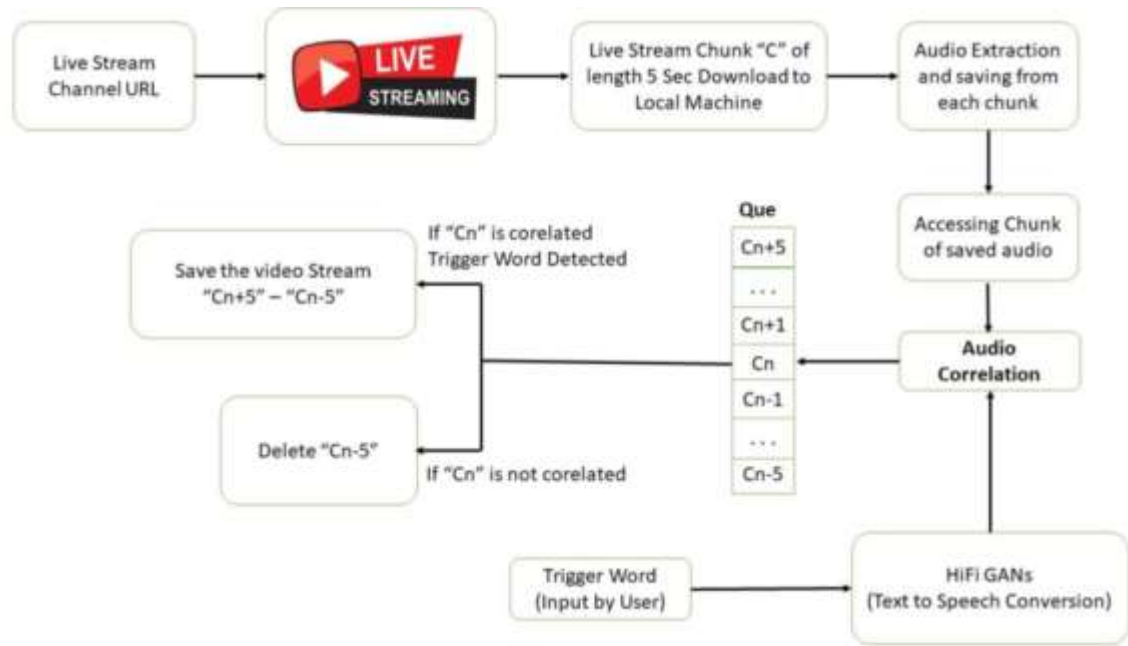


Figure 4 Flow Diagram of Signal-to-Signal Matching for in VMS

To further enhance the system's ability to process continuously streaming video data in real-time, multiple threads for concurrent processing were incorporated. The system is divided into three main threads:

1. **Thread#1 - Live Video Streams Capturing:** In the first thread, continuous 5-second chunks of the live streaming video were downloaded and stored in a queue for further processing.
2. **Thread#2 – Video-to-Audio Conversion:** The second thread converted these downloaded video chunks into audio format to prepare them for matching and analysis.
3. **Thread#3 - Speech Synthesis and Correlation Identification:** The third thread generated the synthesized word using the HiFiGAN model, similar to the previous procedures. The

synthesized word was then matched with the audio chunk from the live streaming video using correlation analysis. If a match was found, indicating the presence of the specific word, the matched chunk and the 5 chunks before and after it were saved. If no match was found, the chunk was removed from the queue.

This enhanced system facilitates real-time processing and matching of live streaming video data, allowing for the extraction of specific word segments from the continuous video stream. As mentioned above, the prototype employs a total of three threads for its operation, as shown in Figure 5. The multithreaded process begins with the input of a specific channel's live streaming URL, which activates the first thread responsible for continuously downloading the live streaming audio. Subsequently, the second thread converts the live streaming video into audio format. Target words input by the user is converted into speech using text-to-speech synthesis, supporting the generation of different dialects within a language. The third thread's subthreads are determined by the number of audio segments synthesized. Each subthread compares the live stream's audio to the target word, and if the correlation surpasses a certain threshold, the saving mechanism is initiated. All subthreads run concurrently, and when the desired threshold is reached, all subthreads pause and then restart.

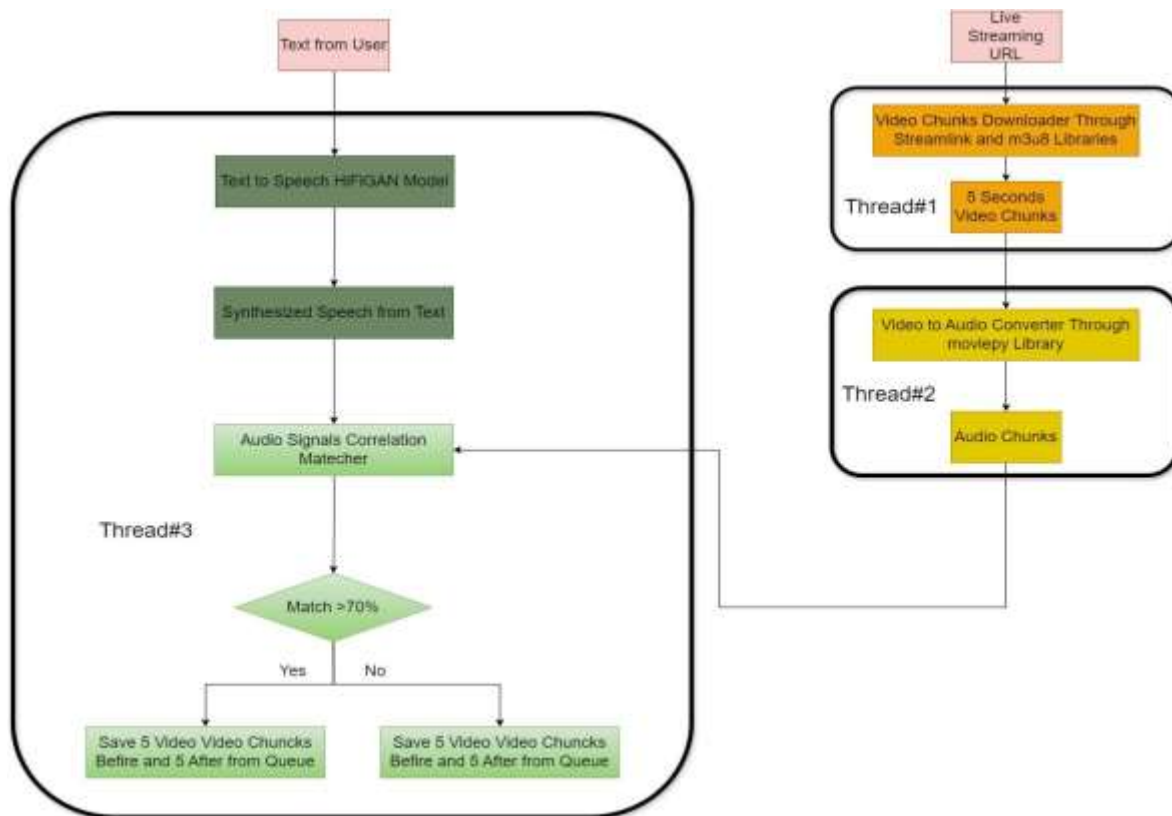


Figure 5 A Multi-threaded Approach for VMS Implementation

Multithreading allows the system to perform multiple operations concurrently, rather than sequentially. In the provided implementation, separate threads can handle different tasks such as downloading video chunks, processing audio files, and performing text matching simultaneously. This parallelism reduces the overall processing time and accelerates the workflow, especially when dealing with large datasets or real-time video streams. By distributing tasks across multiple threads, the system can make better use of available CPU resources. Modern processors have multiple cores that can execute threads concurrently. Multithreading ensures that these cores are utilized effectively, preventing CPU idling and improving the overall throughput of the system. In our case real-time processing is crucial, such as live video analysis or real-time transcription, a multithreaded approach ensures that the system remains responsive. Tasks like downloading video chunks and processing audio can run in the background while the main thread handles user interactions or other high-priority tasks. This responsiveness is essential for applications that require real-time feedback or continuous monitoring.

This system for real-time matching on live streaming videos demonstrates the capability to efficiently process and analyze live streaming content, enabling the extraction of valuable information based on user-defined criteria. This methodology ensures that specific segments of interest within a continuous video stream are accurately identified and saved, facilitating effective real-time content monitoring and analysis. The next section discusses details of each thread.

2. Thread#1 - Live Video Streams Capturing

Live video streams capturing refers to the process of extracting and downloading live video streams from platforms like YouTube. To achieve this, a specific methodology is implemented using open source libraries. The process can be broken down into the following steps:

1. *Utilizing "streamlink" library:* This library allows the piping of streams from various services, including YouTube, into a video player that can be accessed and downloaded locally. It serves as the initial step in the methodology.
2. *Using the "m3u8" library:* Once the stream is accessed, the methodology utilizes the *m3u8* library to load the associated *m3u8* playlist file. The playlist contains important information about the video chunks, such as their URLs and timestamps.

3. *Retrieving the upload chunk URL:* The *get_stream* function is employed to extract the upload chunk URL from the *m3u8* playlist. This URL is necessary to gather the required information for downloading each individual chunk.
4. *Downloading the video chunks:* The *dl_stream* function performs the downloading process. It takes parameters such as the URL, desired filename, and the number of chunks to be downloaded. The function iterates over the specified number of chunks, extracting each one and saving it as a separate video file in the specified directory. The *urllib* library is used for this purpose.
5. *Handling Timestamps and Avoiding Duplicates:* To prevent the downloading of duplicate content, the implemented approach records the timestamp of each downloaded chunk.
6. *Duration of Downloaded Video Segments:* It's important to note that the video data collected from YouTube consists of 5-second segments. These segments continue to be downloaded in real-time until the process is intentionally stopped.
7. *Quantitative Details and Analysis:* The process of capturing live stream videos involves the dynamic and real-time extraction of video content from YouTube. Notably, the implemented approach runs continuously, downloading video segments in real-time until it is manually halted by the user or disrupted by an internet connection issue. As long as the capture process is active, the system consistently acquires and stores 5-second video segments directly from YouTube's live stream. This uninterrupted data flow ensures that users can access a comprehensive and current collection of video content from their selected source.

During the extraction process, several challenges may arise, including network connectivity issues, ensuring the availability of the YouTube video stream, and efficiently handling the downloading and storage of large video chunks. The methodology tackles these challenges by implementing techniques such as exception handling (e.g., handling `TypeError`) to address potential errors during the download process. Overall, the implemented methodology leverages libraries like *streamlink* and *m3u8* to extract live video streams from YouTube. It effectively handles challenges related to network connectivity, ensures successful download and storage of video chunks, and facilitates the extraction of YouTube videos for further analysis and interaction.

3. Thread#2 – Video-to-Audio Conversion

The section of the methodology we are discussing focuses on the important task of extracting audio from downloaded YouTube video chunks and saving them as individual audio files in the WAV format. The process involves the installation and importation of necessary libraries, such as *VideoFileClip* from *moviepy.editor*, *librosa*, *os*, and other supporting libraries. These libraries provide functionalities for video and audio processing, file manipulation, and system operations.

Initially, the proposed methodology defines two paths: *audiopath* and *videopath*. *audiopath* represents the directory where the extracted audio files will be saved, while *videopath* represents the path to the video files that have been downloaded in chunks. To extract audio in the WAV format from the downloaded YouTube video chunks, the following steps are adopted:

1. *Accessing the video file*: The video file is accessed by combining the *videopath* and the current value of the count variable, which represents the current video chunk being processed.
2. *Checking video file existence*: If the video file exists, a *VideoFileClip* object is created using the video file. This object provides access to the video's audio track.
3. *Defining the audio filename and path*: The filename and path for the audio file are defined by appending the current value of the count variable to the *audiopath* directory. This ensures that each extracted audio file has a unique name.
4. *Extracting audio from the video clip*: The audio is extracted from the video clip using the *write_audiofile()* function, which is part of the *VideoFileClip* object. This function saves the extracted audio as a WAV file at the specified location.
5. *Incrementing the count variable*: The count variable is incremented to move on to the next video chunk, ensuring that the correct video file is accessed in the subsequent iteration of the loop.

The implemented methodology utilizes a loop that continuously calls the *process_videos()* function. This loop iterates through the video chunks, performing the audio extraction and saving process for each chunk. If any error occurs during the process, the code handles it by printing an error message and attempting to read the video file again. By executing this code, the YouTube video chunks are processed one by one, and their corresponding audio is extracted and saved as separate WAV files. This step prepares the audio data for further analysis and potential utilization in subsequent stages of the project, such as speech synthesis or other forms of audio processing.

4. Thread#3 - Speech Synthesis and Correlation Identification

In this section, first we will talk about the base algorithm behind the speech synthesis functionality of our VMS. The other section describes the algorithm of correlation identification.

4.1 Speech Synthesis

The user-provided trigger words (those words that need to be detected in the live stream) undergo a transformation into speech before they can be correlated with the live stream YouTube videos being downloaded in real time. To accomplish this, a text-to-speech (TTS) synthesis technique is employed in the project, leveraging the capabilities of HiFiGANs, which are generative adversarial networks (GANs) specifically designed for audio generation. HiFiGANs utilise advanced GAN architecture to produce high-quality speech waveforms that closely resemble natural human speech. It comprises of two main components, as shown in Figure 6:

1. **Generator:** This network creates realistic audio data.
2. **Discriminator:** This network evaluates the authenticity of the audio data distinguishing between real and generated samples.

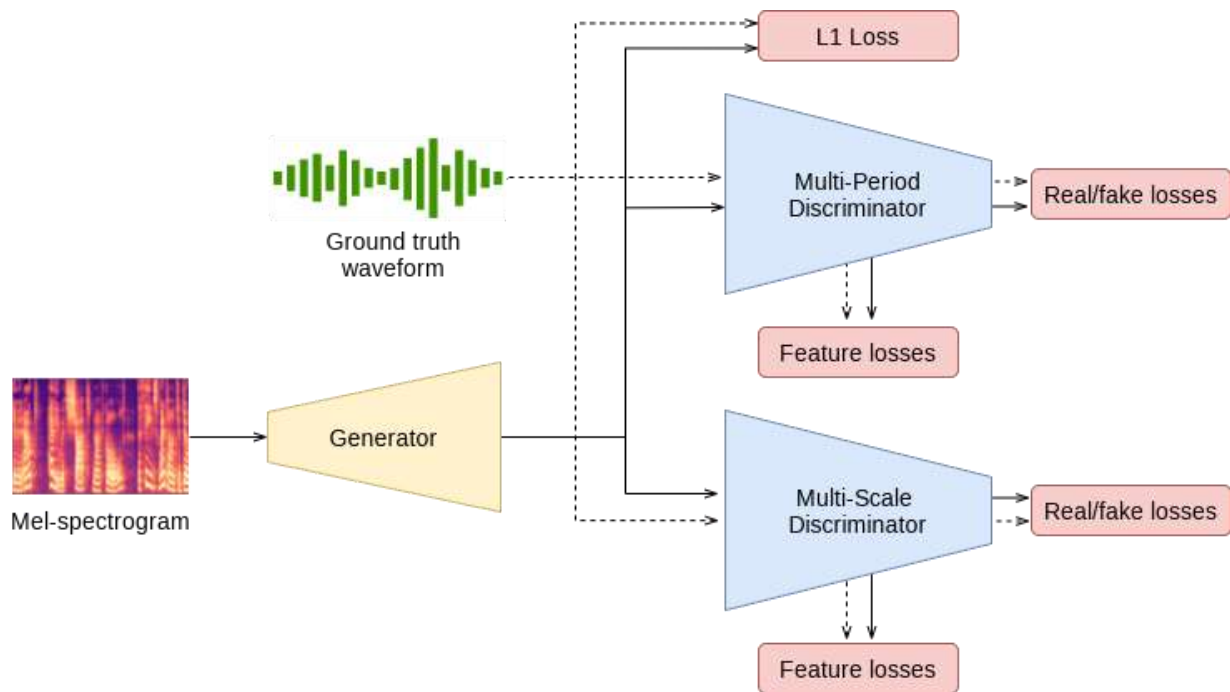


Figure 6 The HiFiGAN Architecture [Kong et al. (2021)]

Let's explore working of each component.

1. Generator:

The generator G in HiFiGAN is responsible for producing audio waveforms from noise or latent vectors. The generator's function can be represented mathematically as:

$$\hat{x} = G(z)$$

Here:

- $\hat{x}$ is the generated audio
- G is the generator network
- z is the input noise or latent vector.

The generator's goal is to create audio $\hat{x}$ that is indistinguishable from real audio.

2. Discriminator:

HiFiGAN uses multiple discriminators to assess the quality of the generated audio. These discriminators operate at different scales and periods to capture various aspects of the audio signal. The discriminators aim to distinguish between real audio samples x and generated audio samples $\hat{x}$.

Each discriminator L_{D_k} tries to maximize the following objective:

$$L_{D_k} = \mathbb{E}_{x \sim p_{data}(x)} [\log D_k(x)] + \mathbb{E}_{\hat{x} \sim p_G(\hat{x})} [\log(1 - D_k(\hat{x}))]$$

In simpler terms:

- The first part $\mathbb{E}_{x \sim p_{data}(x)} [\log D_k(x)]$, measures how well the discriminator can identify generated audio. $\mathbb{E}$ denotes the expected values, and $x \sim p_{data}(x)$ means x is sampled from the real data distribution.
- The second part $\mathbb{E}_{\hat{x} \sim p_G(\hat{x})} [\log(1 - D_k(\hat{x}))]$, measures how well the discriminator can identify generated audio samples. $\mathbb{E}$ denotes the expected values, and $\hat{x} \sim p_G(\hat{x})$ means x is sampled from the distribution of generated audio.

As mentioned above, the discriminator's objective is to maximize L_{D_k} , which involves correctly classifying real and generated audio. The goal is for (x) to be close to 1, meaning the discriminator correctly identifies real samples as real, or in other words, for $(\hat{x})$ the goal is to be close to 0, meaning the discriminator correctly identifies generated samples as fake.

HiFiGAN employs multiple discriminators at different scales and periods to better capture the nuances of the audio signal. This multi-scale and multi-period approach ensures that the model learns to produce realistic audio across different levels of detail and temporal patterns.

In the project, a pre-trained HiFiGAN model is utilised to synthesise speech from the text entered by the user, as shown in Figure 7. The user's input text is encoded and fed into the HiFiGAN model, which then generates an audio waveform that represents the synthesised speech in seven accents: 2 types of US male and female accents, Canadian male accent, Scottish male accent and Indian male accent. The snippet below in Figure 8 shows the testing of speech synthesis in 7 accents.

```
!pip install soundfile transformers datasets sentencepiece
from IPython.display import Audio
from transformers import SpeechT5Processor, SpeechT5ForTextToSpeech, SpeechT5HifiGan
from datasets import load_dataset
import torch
import random
import string
import soundfile as sf
device = "cuda" if torch.cuda.is_available() else "cpu"
# load the processor
processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_tts")
# load the model
model = SpeechT5ForTextToSpeech.from_pretrained("microsoft/speecht5_tts").to(device)
# load the vocoder, that is the voice encoder
vocoder = SpeechT5HifiGan.from_pretrained("microsoft/speecht5_hifigan").to(device)
# we load this dataset to get the speaker embeddings
embeddings_dataset = load_dataset("Matthijs/cmu-arctic-xvectors", split="validation")
# speaker ids from the embeddings dataset
# bdl (US male) slt (US female) jmk (Canadian male) awb (Scottish male)
# rms (US male) clb (US female) ksp (Indian male)
speakers = {
    'awb': 0, # Scottish male
    'bd1': 1138, # US male
    'clb': 2271, # US female
    'jmk': 3483, # Canadian male
    'ksp': 4535, # Indian male
    'rms': 5667, # US male
    'slt': 6799 # US female
}
```

Figure 7 Speech Synthesis with HiFi GAN

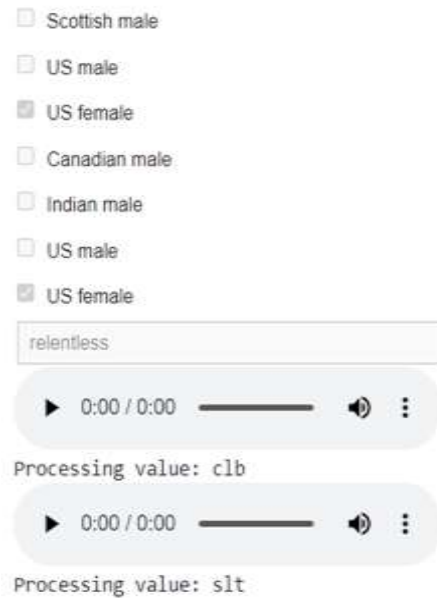


Figure 8 Example of Speech Synthesis given Text and Speakers

Next, the output of the HiFiGAN model and that of Thread#2 are matched for similarity.

4.2 Correlation Identification

The synthesised speech is then utilised in correlation with the real-time audio signal extracted from the YouTube videos as a result of Thread#2. By employing text-to-speech technology and utilising specialised architectures like HiFiGANs, the user's word is converted into an audio representation. This audio representation can be directly correlated with the audio signal obtained from the live YouTube stream. By enabling this correlation between the synthesised speech and the real-time audio signal, the project facilitates the analysis and identification of specific words within the live YouTube stream. The utilisation of HiFiGANs and the conversion of user input into audio form enhance the ability to accurately correlate and compare the user's words with the extracted audio signals, enabling more precise and effective processing and analysis of the YouTube videos.

The correlation process involves comparing the user-entered word, converted into speech, with the extracted audio signal from the live video chunks. This comparison aims to identify any similarity or matching patterns between the two signals. If a significant correlation (optimally set as 70% to avoid making the detection process over sensitive or less sensitive towards audio signals) or match is detected between the user-entered word and the extracted audio signal, it indicates that the

specified word is present in the video stream. The snippet below in Figure 9 shows the testing of correlation identification with a threshold of 70%:

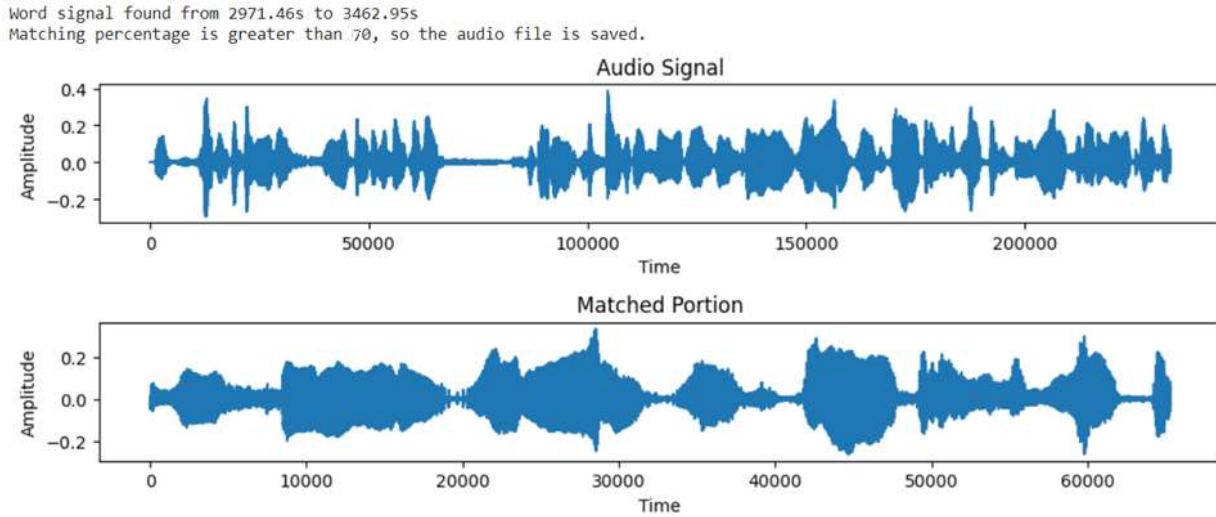


Figure 9 The Process of Correlation Identification

The formula used to calculate the correlation is given below:

$$\text{correlate}(a, v, \text{mode} = \text{valid}) \sum_{i=0}^{n-1} a_i v_{i-j}$$

Where, n is the length of a (audio signal of context as NumPy array).

This formula can be explained as follows:

- The summation is over the elements of a , starting from the beginning.
- At each step, we multiply the element of a with the corresponding element of v (Audio signal of target as NumPy array), shifted by j positions.
- The sum of these products is the correlation value at lag j .

The valid mode means that the output correlation sequence will be the same length as the shortest input sequence, minus one. This is because the correlation at lag j cannot be calculated unless both input sequences have at least $j + 1$ elements. Upon identifying a match, the algorithm captures a segment of the audio, including 25 seconds before and 25 seconds after the identified search term. This segment is then saved for further analysis. To manage storage and processing efficiently, in

first 50 seconds the 25 seconds of the audio and video data that do not contain a match are discarded, followed by discarding every subsequent 5-second chunk to maintain a continuous 25-second buffer if the word is found after that. The deletion and maintenance of audio and video is achieved through the employment of threading method. This process of checking 5-second chunks and updating the saved segments continues dynamically, ensuring that the system remains efficient while maintaining a high degree of accuracy in signal matching.

By employing this correlation-based approach, the proposed methodology aims to accurately identify specific words within the live video stream. It takes appropriate actions based on the presence or absence of the specified words. This correlation process enhances the efficiency and effectiveness of the subsequent analysis and processing stages, allowing for targeted investigation and utilization of the extracted video chunks. Moreover, we introduced a multi-channel and multi-search word facility through threading. This enhancement allows the system to handle multiple channels and search terms simultaneously, significantly improving the system's versatility and responsiveness. By processing different channels and search terms in parallel, we ensure that our approach can efficiently handle the demands of a dynamic and diverse set of inputs.

By implementing the above three threads, we successfully extract the video content of a word that the user is intended to search from the live video streams in English language. However, it is important to note that the language-agnostic problem was not fully resolved in this implementation and we were not able to perform the video extraction for Urdu language. An obvious reason for this is that the HiFiGAN model was trained on specific languages especially English and might not generalize well to all languages. Further research and experimentation would be required to address this limitation and could be considered as an extension of this project.

In the next chapter, we will discuss the experiments that we performed during the process of implementing the process and last but not least, we outline the implementation of Graphical User Interface (GUI).

Chapter 4 – Experimentation and Interfacing

This section delves into the detailed experiments conducted as part of the project, and the resulting outcomes that were discussed with the project team at NESCOM to refine the initial prototype. The insights gained from these experiments have helped us align the prototype more effectively with the desired objectives of the project. Further, interfacing for interacting with the Video Monitoring System (VMS) is developed which is outlined in this chapter.

1. Experiments and Results

In this section we will talk about the different experiments that has been done during prototyping stage to finalize the method to be used for the final product development of Video Monitoring System (VMS).

1.1 Speech-to-Text Conversion/Transcription-based Approach (English Language)

The first approach employed in this project is based on a transcription-based search mechanism. As shown in Figure 10, this method begins by converting video content into an audio format. The audio is then processed using an end-to-end transcription model, specifically the transformer-based Wav2Vec2, which we trained on the TIMIT Acoustic-Phonetic Continuous Speech Corpus. This model generates a time-stamped transcription of the audio, capturing both the text and the corresponding time intervals.

Once the transcription is obtained, the text is scanned for the presence of the search term. If the search term is found, the algorithm retrieves the timestamp associated with the occurrence of the word. To provide context, the system also captures the audio and corresponding video segment from 30 seconds before to 30 seconds after the identified search term. This approach allows for precise localization of the search term within the video content. The implementation of this approach involved several steps which are detailed below:

1. Firstly, the audio was extracted from the video source using video processing libraries, separating the audio track from the visual content.

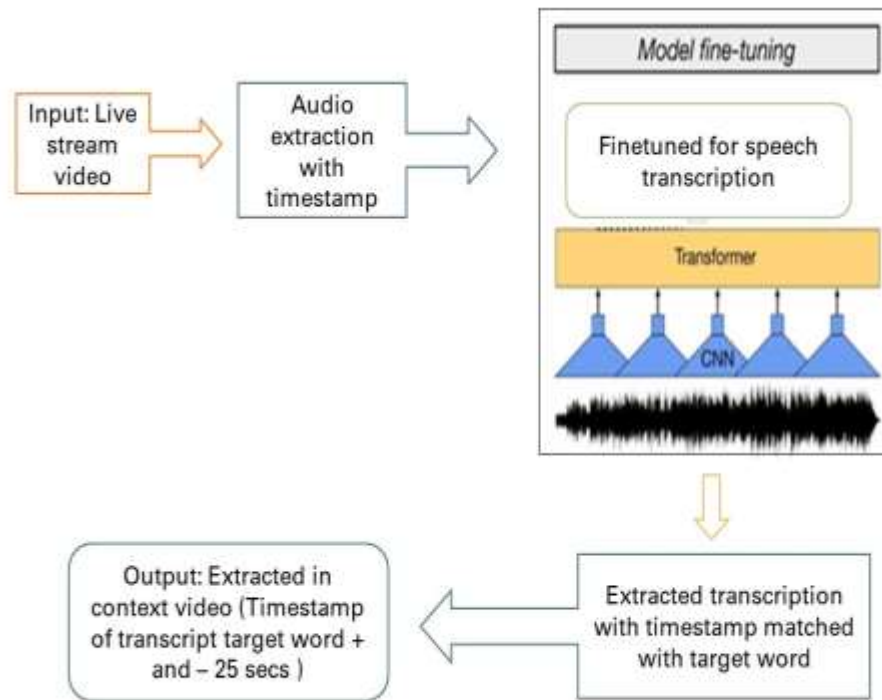


Figure 10 Speech-to-Text Conversion-based Approach

2. Next, an English Automatic Speech Recognition (ASR) system was implemented utilizing the wav2vec2 model. This state-of-the-art model, trained through self-supervised learning techniques, offered high accuracy in speech recognition.
3. The extracted audio was passed through the ASR system, leveraging the pre-trained weights of the wav2vec2 model. These weights were trained on a substantial corpus of English audio data, enabling the model to convert spoken language into textual form. The ASR system generated a transcription of the spoken words from the video's audio data.
4. Following transcription, text matching techniques such as exact string matching were employed to search for the specific word within the transcribed text. This process allowed for the identification of the desired word within the transcription.
5. Once the specific word was identified, the corresponding timestamp of its occurrence was recorded. This timestamp represented the location of the word within the audio timeline of the video.
6. Using the obtained timestamp, the specific part of the video containing the desired word was extracted. This could be achieved by trimming the video based on the start and end timestamps or by creating a separate video clip that included the specific segment.

The above implementation successfully extracted the specific part of the video where the desired word was spoken. The wav2vec2 ASR model achieved an accuracy rate of 75%, on TIMIT dataset, in transcribing the English audio, demonstrating its effectiveness in accurately converting speech into text. We then started the experiments on Urdu language, which is detailed in the next section.

1.2 Speech-to-Text Conversion/Transcription-based Approach (Urdu Language)

To address the limitation of language dependency, this experiment involved training a model for the Urdu language dataset shown in Figure 11. The intention was to expand the system's capabilities to capture and extract specific parts of videos in multiple languages.

	transcription
0	اس سے دعوت کو نقصان پہنچتا ہے۔
1	اب ہر شعبے کے ماہرین تو ہمیں مل جاتے ہیں مگر ان
2	کوئی بحث ہمارے ملازمین کی صحت اور حفاظت سے زیادہ اہم نہیں
3	رہے جیسے بڑے لوگوں کے ہوتے کبھی کسی کو انصاف ملا ہے نہ ملے گا
4	بہت سے لوگ محض اپنا وطن اپنی جان سے پیارا
5	یہ بات جاپان میں ہونے والی ایک طبی تحقیق میں سامنے آئی ہے
6	پاکستانی معائنے کا بیلنس ٹوٹ گیا۔
7	شہانہ کی کارڈ کی تصدیق ہونے کی صورت میں سکرین پر ٹک کا نشان سامنے آ جائے گا
8	یہ لوگ کسی کو کیا منہ دکھائیں گے؟
9	یہی کچھ ہمیں دیکھنے کو مل رہا ہے۔

Figure 11 Urdu Dataset Speech with Transcription

To fulfil the requirement, this experiment involved the fine-tuning of the wav2vec2 model on a specifically collected Urdu language dataset. The training process encompassed updating the model's weights using the Urdu dataset and optimizing it to enhance its understanding and transcription capabilities for Urdu speech. The fine-tuning procedure was conducted over 70 epochs shown in Figure 12 to ensure that the model obtained sufficient exposure to the Urdu language data. Throughout the training, the model's parameters were iteratively adjusted to improve its accuracy in transcribing Urdu audio.

Step	Training Loss	Validation Loss	Wer	Cer
300	3.771600	1.791788	0.853038	0.384550
600	0.724300	1.828540	0.818608	0.403253
900	0.553400	1.881453	0.830633	0.390388
1200	0.496300	1.960695	0.788987	0.373215
1500	0.490600	1.862623	0.822658	0.382368
1800	0.505100	1.828654	0.804304	0.366470
2100	0.459500	1.831752	0.817595	0.373753
2400	0.445300	1.746694	0.804684	0.361284
2700	0.414900	1.679488	0.780633	0.365762
3000	0.401700	1.675084	0.770000	0.350346
3300	0.372400	1.810464	0.760000	0.353378
3600	0.358900	1.657724	0.771392	0.355645
3900	0.359800	1.697178	0.761013	0.351253
4200	0.363200	1.700443	0.752532	0.346548
4500	0.347100	1.693250	0.745063	0.345642

Figure 12 Training of Wav2vec2 on Urdu Dataset

Following the training process, the performance of the fine-tuned Urdu Automatic Speech Recognition (ASR) model was evaluated. The evaluation aimed to measure the accuracy of the model in accurately converting Urdu speech into textual form. The assessment of our fine-tuned Urdu ASR model showcases impressive results, with an achieved accuracy of 85% in terms of Character Error Rate, as illustrated in Figure 13. It's important to note that Word Error Rate (WER) can be significantly higher than Character Error Rate (CER) due to the presence of homophones in transcription. In WER, if even a single character is incorrect within a word, the entire word is marked as inaccurate. In contrast, CER evaluates individual characters, allowing for a more nuanced assessment, where only the specific character in error is considered, rather than labelling the entire word as incorrect. This outcome indicated the effectiveness of the model in accurately converting spoken Urdu language into written text. Comparison of Word error rate and character error rate is shown in Figure 6 below.

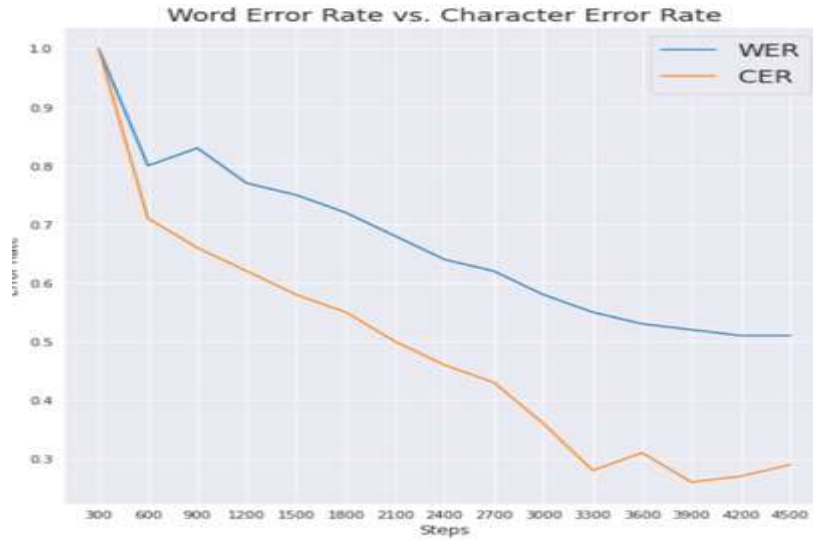


Figure 13 WER vs. CER

During the evaluation, a problem emerged concerning the consistent identification and differentiation of Urdu homophones by the model. Urdu homophones, which are words with the same pronunciation but different meanings, posed a challenge in accurately capturing the specific word, especially in cases where Urdu homophones were present shown in Figure 14. This limitation impacted the model's ability to distinguish between multiple meanings of the same spoken word. Additionally, the system's language-agnostic capability remained unresolved. Although the model was fine-tuned specifically for Urdu, the limitations associated with language dependency still persisted. We believe that the solution to this project required extensively data collection for Urdu language. Due to limitation of time and scope of the project, we decided to keep up our scope to English language and because of this we opted the approach of audio signal-to-signal matching, so that language dependency can be reduced up to some extent.

	pred_str	text
0	اس لیے اسے اسی حیثیت سے پیش کیا جائے گا۔	اس لیے اسے اسی حیثیت سے پیش کیا جائے گا۔
1	نو سپریم کورٹ	نو سپریم کورٹ۔
2	سول ب ہتقدم دے رہا	سفر بہت آرام دہ رہا۔
3	نوبے فطان کوئی بی بی۔	نو پیر کپتان کوئی بی بی۔
4	میرے لیے اس سارے کہیں میں بی بی بات باعث تشویش ہے۔	میرے لیے اس سارے کہیں میں بی بی بات باعث تشویش ہے۔

Figure 14 Urdu homophones Problem

Comparative analysis of the two approaches, speech-to-text and audio signal-to-signal matching was further done by using a custom dataset TIMIT.

1.3 Algorithms Comparative Evaluation

Comparison between algorithms is detailed in this section. But before moving on to that we want to explain the dataset used for the comparative analysis of the two algorithms from the perspective of performance.

1. The TIMIT Dataset:

The TIMIT dataset, developed by the Linguistic Data Consortium (LDC) in collaboration with Texas Instruments and the Massachusetts Institute of Technology (MIT), is a seminal resource in the field of speech processing and phonetics. It was created to facilitate research and development in automatic speech recognition (ASR), speaker recognition, and linguistic studies by providing a standardized corpus of phonetically and acoustically annotated speech data.

TIMIT comprises recordings from 630 speakers representing eight major dialect regions of the United States. Each speaker recorded ten phonetically rich sentences, resulting in a total of 6,300 utterances. Key features of the dataset include:

1. *Phonetic Transcriptions:* Every utterance in TIMIT is manually annotated with phonetic transcriptions using the International Phonetic Alphabet (IPA), enabling precise analysis of speech sounds at the phonemic level.
2. *Acoustic Recordings:* The dataset includes 16-bit, 16 kHz PCM-encoded WAV files, ensuring high-quality audio recordings suitable for various computational analyses.
3. *Speaker Information:* TIMIT provides demographic details such as gender, age, and geographical origin for each speaker, facilitating studies on speaker variability and dialectal differences.

2. Signal-to-Signal Matching Approach:

We implemented the signal-to-signal matching approach on a Jetson Nano device with 4 GB RAM. The device was capable of generating six accents for the search word using HiFiGAN, which

utilized up to 1 GB of RAM. We ran six threads for accent generation simultaneously with the signal matching process, including the deletion process and saving of in-context video.

To evaluate the accuracy of the signal-to-signal matching approach, we extracted a small test dataset from the TIMIT dataset. Our tests showed an accuracy of 72%. This implementation leveraged the Jetson Nano's GPU for parallel processing, which allowed us to run the system efficiently despite the limited resources.

3. Transcription-Based Approach:

In contrast, the transcription-based approach was implemented using Google Colab with a graphical user interface (GUI). This method could not be implemented on the Jetson Nano due to resource constraints, specifically RAM and GPU limitations. Transcription-based models generally require significantly more RAM because speech transcription methods are resource-intensive.

We tested the transcription-based approach on a custom test set from the TIMIT dataset and achieved an accuracy of 75%. The higher resource requirements and complexity of the transcription-based approach made it unsuitable for deployment on the Jetson Nano, necessitating the use of a more powerful platform like Google Colab.

Table 3 Comparison of Algorithms

Aspect	Signal-to-Signal Matching	Transcription-Based Search
Platform	Jetson Nano (4 GB RAM)	Google Colab
RAM Usage	Up to 1 GB for HiFiGAN Remaining is efficiently utilised for multichannel and multi word search	More than 8 GB required
Parallel Processing	Utilised GPU for multiple threads	Not implemented on Jetson Nano
Test Dataset	TIMIT custom test set	TIMIT custom test set

Accuracy	72%	75%
Feasibility on Jetson Nano	Yes	No

While the transcription-based method showed a slightly higher accuracy, the signal-to-signal matching algorithm was more feasible for deployment on the Jetson Nano due to its efficient use of resources and ability to leverage GPU for parallel processing. This comparison in Table 3 highlights the trade-offs between resource constraints and algorithmic accuracy in different deployment environments. Further as highlighted in the previous section that we desire our system to be language independent, which is supported by signal-to-signal matching approach, so because of these two reason we finalized the signal-to-signal matching approach and our methodology mentioned in Chapter 3 is based on that.

2. The Graphical User Interface (GUI)

The graphical user interface (GUI) of the system is developed using *Tkinter*, the Python library known for its simplicity and ease of use in creating user interfaces. Within the GUI, users are prompted to input trigger words, which can be single word or multiple words separated by commas, in the 'Search Word' field. Additionally, users are required to provide the URL of the live stream of the YouTube video they wish to analyze. Upon pressing the *Start* button, the system initiates it's processing, correlating the provided trigger words with the real-time audio stream from the specified YouTube video. A *View* button is provided within the interface, facilitating the listing of all videos in which triggered words were detected, stored in a designated folder for easy access and review. Moreover, a *Status* indicator dynamically showcases the current task being performed in real-time, offering user's insights into the system's ongoing operations. Through this intuitive GUI, shown in Figure 15, users can conveniently interact with the system, initiating analyses and accessing results with ease.

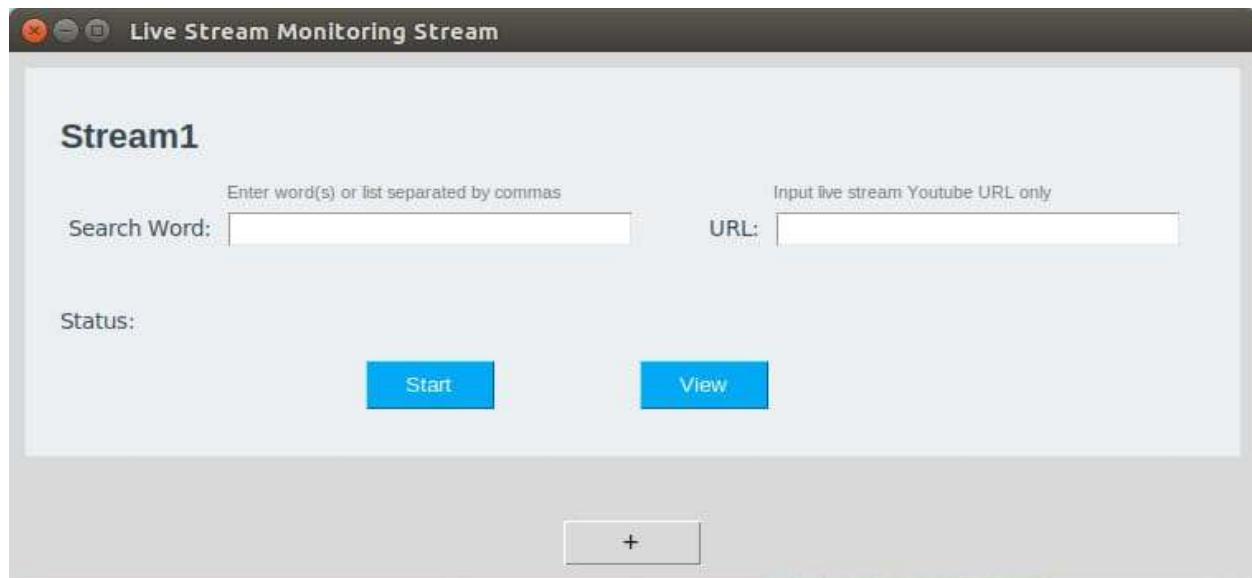


Figure 15 The GUI of VMS

Within the system's interface, key information is provided to enhance user understanding and monitoring of ongoing processes. The *Created On* timestamp displays the date and time when processing commenced for a particular live stream, offering users insight into the temporal context of analysis. Alongside, the *Channel* field succinctly identifies the name of the news channel whose live stream is currently under monitoring, aiding users in tracking specific sources of content. Additionally, the *Detection* metric serves as a valuable indicator, showcasing the total number of detected trigger words within the live streams, enabling users to gauge the prevalence and significance of identified content. These informative elements, as shown in Figure 16, collectively empower users with essential context and real-time feedback, facilitating informed decision-making and comprehensive oversight of the system's operations. Integrating a user-friendly experience, the interface introduces a handy feature with the + button. When clicked, this button smoothly expands the interface, making room for adding a second live stream channel for monitoring. Users can easily switch between channels, boosting their ability to analyze multiple content sources simultaneously. This user-friendly feature fosters efficiency and adaptability in content monitoring, enabling users to broaden their analysis scope effortlessly. Whether comparing content from various news channels or conducting parallel investigations, the addition of the + button enhances the system's versatility, empowering users with greater control and insight into multimedia streams.

Content Extraction and Filtering from Video Stream

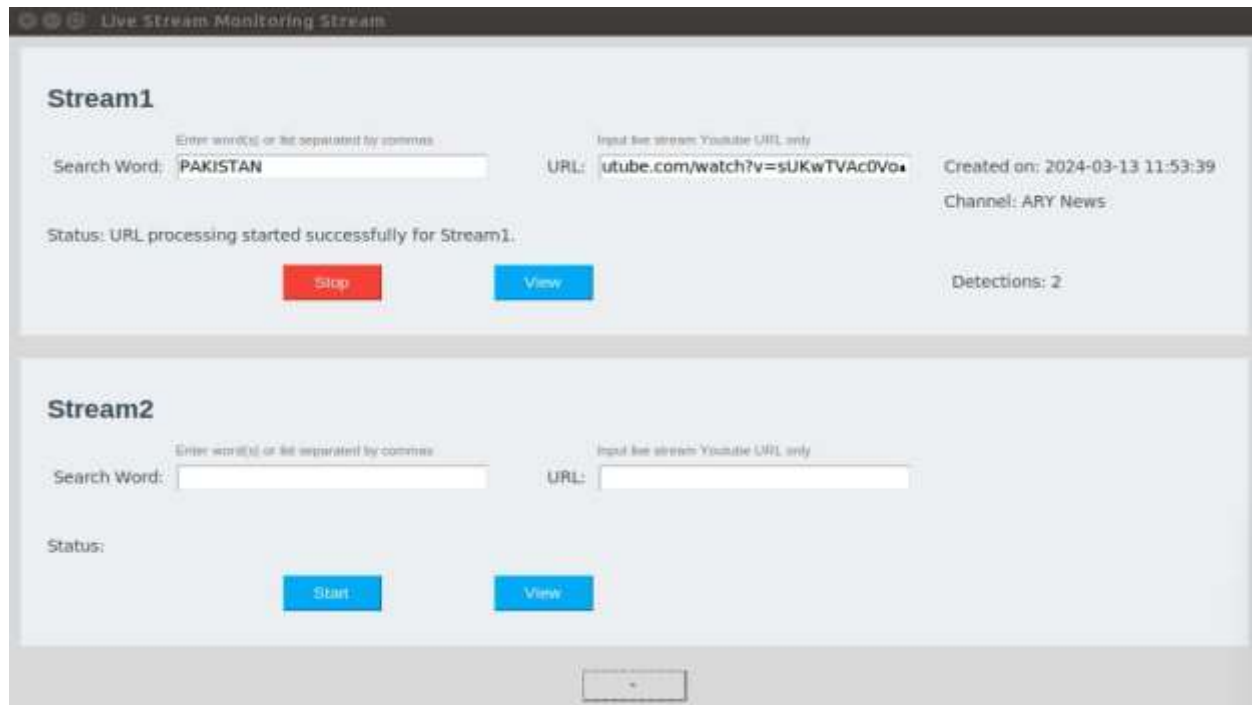


Figure 16 GUI Providing Real-time Feedback Provided to the User

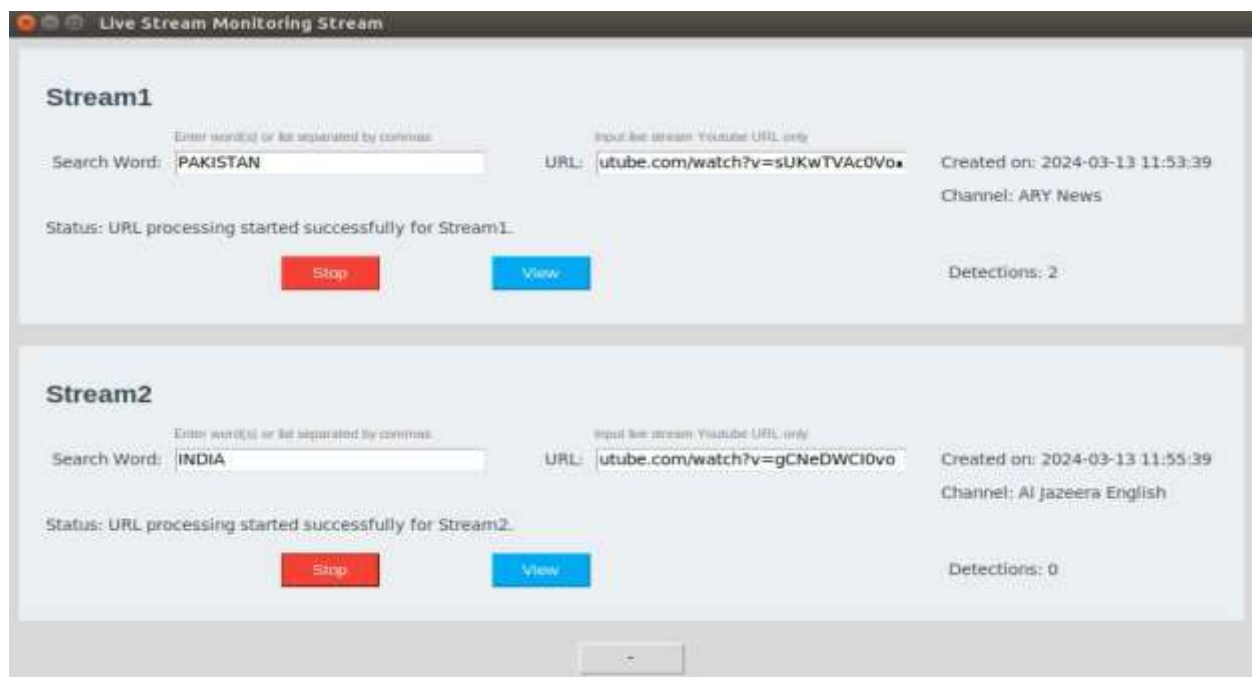


Figure 17 GUI Providing Dual Channel Support

The screenshot shown in Figure 17 demonstrates the full functionality of our system, with both streams operating seamlessly in parallel. It showcases our system's ability to multitask effectively,

providing comprehensive monitoring across multiple channels simultaneously. This underscores our commitment to delivering efficient and insightful real-time analysis.

3. Tools & Techniques Used

In the development of our system, we have utilized a range of programming tools and techniques tailored to meet the specific requirements of our project. Python serves as the primary programming language, chosen for its versatility, extensive libraries, and ease of integration with external tools and APIs. Within Python, we leverage the Tkinter library for GUI development, providing users with an intuitive interface for interacting with our system. Additionally, we employ advanced machine learning techniques, such as HiFiGANs (High Fidelity Generative Adversarial Networks), to synthesize audio from user-provided text inputs. This synthesis process enables the generation of lifelike speech waveforms closely resembling natural human speech, facilitating accurate identification and analysis of specific trigger words within live audio streams.

Furthermore, our system harnesses the power of NVIDIA's Jetson Nano for efficient data processing and analysis tasks. The Jetson Nano, equipped with a quad-core Arm A57 CPU and a Maxwell GPU, provides the computational horsepower needed for real-time video stream capturing, audio signal processing, and AI-based content analysis. Its compact form factor and low power consumption make it an ideal choice for deployment on edge devices, ensuring seamless operation even in resource-constrained environments.

In terms of system specifications, our solution offers robust functionality for word searching and content extraction from live audio/video streams. Users can input search words or phrases, separated by commas, along with the URL of the YouTube live stream they wish to analyze. The system then generates synthesized audio for each search word using HiFiGANs and proceeds to download live video stream chunks for further analysis. The GUI provides users with real-time status updates on the processing of streams, enhancing transparency and user engagement.

Overall, our system is characterized by its utilization of cutting-edge programming tools and techniques, coupled with efficient system specifications, to deliver a seamless and intuitive platform for word searching and content extraction from live multimedia streams.

Chapter 5 – Conclusion

This chapter concludes the report by first summarizing the work completed in the project. Following the summary, it provides a justification of how the milestones have been achieved. Lastly, the chapter discusses the project's limitations and outlines potential areas for future work.

1. Summary

For the development of the project, we have adopted a prototype-based approach. Based on this approach, the development of the project is divided into the following three stages:

- Stage 1- Initial Prototype Development
- Stage 2 - Integration & Prototype Refinement
- Stage 3 - Packaging and Deployment

Sub-tasks involved in each of these stages are shown in Figure 18 below:

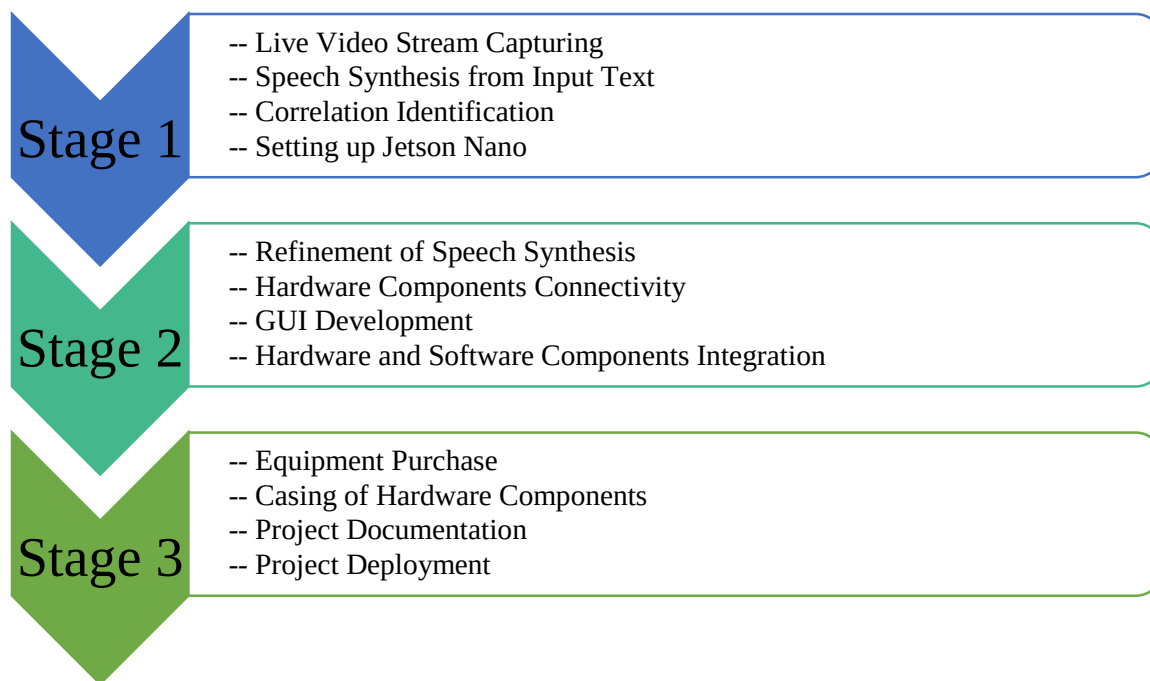


Figure 18 Development Stages of the Project

We are done with the first two stages completely and Stage 3 is in progress. Let's summarize the work done in these stages.

1.1 Stage 1- Initial Prototype Development

This stage focused on developing the foundational elements of the project. The groundwork laid during this stage established the core functionality and prepared the system for further refinement. Key tasks included live video stream capturing, speech synthesis from input text, audio signal matching, and setting up the Jetson Nano hardware.

1. *Live Video Stream Capturing:* The project utilizes streamlink to pipe streams from YouTube into a video player and m3u8 to load playlist files containing video chunk URLs. This enables the continuous download and local storage of video chunks, ensuring uninterrupted and current video content collection.
2. *Speech Synthesis from Input Text:* HiFiGAN models are employed to transform user-provided trigger words into high-fidelity speech. This synthesized speech is used for real-time correlation with the audio extracted from live video streams, ensuring accurate identification of specific words.
3. *Correlation Identification:* The project extracts audio from video chunks, saves them as WAV files, and compares these with the synthesized speech using a correlation-based methodology. Significant matches trigger the saving of relevant video chunks, capturing the context around the detected word.
4. *Setting up Jetson Nano:* The Jetson Nano is set up with essential software and hardware components to facilitate GPU-accelerated computing. This setup ensures efficient processing of live stream extraction, audio processing, and speech synthesis tasks.

1.2 Stage 2 - Integration & Prototype Refinement

In this stage, the project moved towards enhancing and refining the initial prototype. This stage aimed to create a more polished and robust system. Tasks included improving speech synthesis capabilities, ensuring seamless hardware connectivity, developing a user-friendly GUI, and integrating both hardware and software components.

1. *Refinement of Speech Synthesis:* The integration of HiFiGAN enhances the project's capability by producing multiple speech variants and accents, improving the accuracy of speech matching.
2. *Hardware Components Connectivity:* The project involves connecting various hardware components to the Jetson Nano, ensuring efficient operation and interaction between these components.
3. *GUI Development:* A user-friendly interface is developed using Tkinter, allowing users to input search words and YouTube channel URLs, and interact with the system effortlessly.
4. *Hardware and Software Components Integration:* The project combines hardware and software components into a cohesive system named the Video Monitoring System (VMS). This integration ensures seamless coordination and functionality.

1.3 Stage 3: Packaging and Deployment

The final stage involved preparing the system for real-world deployment. The focus was on creating a user-friendly, durable, and fully functional solution ready for practical application. This included purchasing necessary equipment, casing the hardware components, comprehensive project documentation, and deploying the system.

1. *Equipment Purchase:* High-quality components are procured for the final development of the VMS, ensuring reliability and longevity.
2. *Casing of Hardware Components:* The hardware components are securely mounted and organized within an enclosure, ensuring protection and ease of use.
3. *Project Documentation:* Comprehensive documentation is prepared, including system architecture, installation guides, source code documentation, and user manuals, ensuring ease of understanding and maintenance.
4. *Project Deployment:* The VMS is rigorously tested, installed in the intended environment, and user training is provided. A monitoring and support framework is established to address any issues post-deployment.

By following these stages and tasks, the project “Video Monitoring System (VMS)” successfully identifying spoken words, records related content, and builds a box-based solution with integrated AI for word spotting and video stream capturing. This moves just to the justification of the milestones set at the beginning of the project.

2. Contribution of the Project

The system developed as a result of the work done in the project achieves the milestone set at the beginning of the project.

- **Objective 1: Developing a module that can identify the spoken time of a specific word from the video stream**

The project effectively fulfils this objective through the implementation of audio signal-to-signal matching. The methodology includes extracting audio from live video streams, converting the extracted audio to WAV files, and then matching these audio files with user-provided trigger words synthesized into speech. The system utilizes a sophisticated text-to-speech (TTS) synthesis technique powered by HiFiGANs to transform the user input into high-fidelity speech. This synthesized speech is then meticulously correlated with the real-time audio signal extracted from the video streams, allowing the system to identify the precise spoken time of the specific word. The use of advanced HiFiGAN models ensures high accuracy in speech recognition, which is crucial for the precise identification of spoken words within the video stream.

- **Objective 2: Developing a mechanism for recording the contents related to the searched word from the live video stream**

The project addresses this objective by implementing a robust system for capturing and processing live video streams in real-time. The process involves continuous downloading of 5-second video chunks from live streams, converting these chunks to audio, and matching the audio with synthesized speech generated from user-entered words. Upon detecting a significant match, the system records the relevant video segments by saving the chunks surrounding the occurrence of the trigger word. Specifically, the system captures and stores 5 chunks before and after the detected

word, ensuring that the context around the searched word is preserved. This comprehensive approach ensures that all relevant content related to the searched word is accurately recorded from the live video stream.

- **Objective 3: Building a box-based solution having a built-in AI module for word spotting and video stream capturing mechanism**

The project lays a solid foundation for a box-based solution by integrating various advanced technologies and components into a cohesive system. The implementation includes the setup of Jetson Nano for processing, utilizing open-source libraries like streamlink and m3u8 for video stream capturing, and employing HiFiGAN for speech synthesis. The system is designed to operate continuously, capturing live video streams, converting them to audio, and performing real-time audio analysis to spot specific words. The integration of hardware components, such as the Jetson Nano, cooling fans, hard drives, and power supply units, along with the development of a user-friendly GUI, ensures that the system is not only functional but also user-friendly and portable. This modular and scalable architecture can be encapsulated into a standalone box-based solution, providing an efficient tool for real-time word spotting and video stream capturing.

3. Limitations & Future Work

As with any complex project, this video monitoring system has encountered certain limitations and presents opportunities for further enhancement. In this section, we discuss the challenges faced during development and propose future work that can address these limitations and expand the capabilities of the system. By acknowledging these areas, we aim to provide a roadmap for ongoing improvements and innovations in the project.

Following are some of the limitations of this work:

1. *Language Restriction:* The current implementation is limited to English due to challenges faced in speech synthesis, text-to-speech translation, and audio signal-to-signal correlation for other languages, particularly Urdu.

2. *Accuracy Variability:* The accuracy of speech recognition and word spotting may vary based on the quality of the live stream audio, background noise, and the clarity of spoken words. Accents and speech variations might also impact the matching accuracy.
3. *Limited Contextual Understanding:* The system captures video chunks based on audio matches but lacks a deeper contextual understanding of the content, which could be valuable for more comprehensive analysis.

Following points may be considered for the extension of the project in future:

1. *Multilingual Support:* Extend the system to support multiple languages, including Urdu. This involves improving speech synthesis and recognition capabilities for diverse languages and dialects.
2. *Advanced Speech Recognition:* Integrate more advanced speech recognition models and natural language processing (NLP) techniques to improve accuracy in identifying spoken words, even in noisy or complex audio environments.
4. *Contextual Analysis:* Incorporate deeper contextual analysis capabilities to understand the broader context of identified words within video streams. This could involve sentiment analysis, topic modeling, and other NLP techniques to provide richer insights.
5. *User-Friendly Interface Improvements:* Enhance the GUI to provide more features and customization options for users. This includes advanced search capabilities, real-time monitoring dashboards, and more intuitive controls.
6. *Automated Alerts and Reporting:* Develop automated alerting and reporting features that notify users of significant matches or trends in real-time. This could be useful for monitoring live events, news channels, and other dynamic content sources.
7. *Integration with Other Data Sources:* Explore the integration of the system with other data sources and platforms (e.g., social media, news aggregators) to provide a more comprehensive monitoring solution.

By addressing these limitations and focusing on the outlined future work, the project can evolve into a more robust, versatile, and user-friendly solution for real-time video stream analysis and word spotting.

Annex A

6/26/24, 1:08 PM

School of Electrical Engg & Computer Science Mail - Purchase of Components - E117809



Dr Rabia Irfan <rabia.irfan@seecs.edu.pk>

Purchase of Components - E117809

1 message

President Local Purchase Committee <plp@seecs.edu.pk>

Tue, Jun 25, 2024 at 4:13 PM

To: Dr Rabia Irfan <rabia.irfan@seecs.edu.pk>

Cc: Local Purchase Committee <lpc@seecs.edu.pk>

Dear Dr. Rabia Irfan,

It is intimated that purchase order has already been placed. Delivery of items is expected after 3-4 weeks (Import from abroad).

Regards
for PLPC

References

1. Chan, W., Jaitly, N., Le, Q. V., & Vinyals, O. (2016). Listen, Attend and Spell. In Proceedings of the Annual Conference of the International Speech Communication Association (INTERSPEECH) (pp. 496-500).
2. Graves, A., Mohamed, A., & Hinton, G. (2013). Speech recognition with deep recurrent neural networks. In IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (pp. 6645-6649).
3. Gulati, A., Qin, L., Chiu, C. C., & et al. (2020). Conformer: Convolution-augmented Transformer for Speech Recognition. In IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (pp. 7484-7488).
4. Hinton, G. E., Deng, L., Yu, D., Dahl, G. E., Mohamed, A. R., Jaitly, N., ... & Kingsbury, B. (2012). Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. IEEE Signal Processing Magazine, 29(6), 82-97.
5. Ko, T., Peddinti, V., Povey, D., & Khudanpur, S. (2017). A study on data augmentation of reverberant speech for robust speech recognition. In Proceedings of the Annual Conference of the International Speech Communication Association (INTERSPEECH) (pp. 1517-1521).
6. Sak, H., Senior, A., & Beaufays, F. (2015). Long short-term memory recurrent neural network architectures for large scale acoustic modeling. In Proceedings of the Annual Conference of the International Speech Communication Association (INTERSPEECH) (pp. 338-342).
7. Chen, L., Li, W., Xu, Y., & et al. (2019). Deep metric learning for audio-based music similarity. IEEE Transactions on Multimedia, 21(3), 702-714.
8. Choi, K., Fazekas, G., Cho, K., & et al. (2017). A tutorial on deep learning for music information retrieval. arXiv preprint arXiv:1709.04396.
9. Jin, X., Zhang, L., Du, B., & et al. (2020). Graph convolutional networks for audio-based music similarity. IEEE Transactions on Multimedia, 22(1), 146-158.
10. Kong, Q., Yan, J., & Plumbley, M. D. (2021). Audio self-supervised learning: A survey. arXiv preprint arXiv:2103.14510.
11. Kong, J., Kim, J., & Bae, J. (2020). Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis. Advances in neural information processing systems, 33, 17022-17033.
12. Kumar, A., Verma, V., Namboodiri, V., & et al. (2020). Graph spectrogram convolutions for audio matching. In Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (pp. 626-630).

13. Lee, H., Pham, N. T., Nam, J., & et al. (2019). Sample-level deep convolutional neural networks for music auto-tagging using raw waveforms. In Proceedings of the International Society for Music Information Retrieval Conference (ISMIR) (pp. 522-529).
14. Klatt, D. H. (1987). Review of text-to-speech conversion for English. *Journal of the Acoustical Society of America*, 82(3), 737-793.
15. Shen, J., Pang, R., Weiss, R. J., & et al. (2018). Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions. In Proceedings of the 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).
16. Skerry-Ryan, R., Stanton, D., & et al. (2018). Towards end-to-end prosody transfer for expressive speech synthesis with Tacotron. In Proceedings of the 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).
17. Van den Oord, A., Dieleman, S., & et al. (2016). WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*.
18. Zen, H., Tokuda, K., & et al. (2009). Statistical parametric speech synthesis. *Speech Communication*, 51(11), 1039-1064.